INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY(ICDFA)

SBT-DF202 — Computer and Digital Forensics

Lab 4 – Steganography and Hidden Data Detection

Student Name: Abubakari-Sadique Hamidu

Student ID: 2025-FWSD-11392

Instructor: Dr. Aminu Idris (CCNA, CompTIA Security+, CEH, OSCP, CISSP | MPCSEAN — Cybersecurity Educator & Mentor)

5th September 2026

GitHub repo: github.com/Abubakari-Sadique/Assignments-ICDFA/tree/main/forensics/lab4-steganography

Tools and Resources Used

Kali Linux (via Docker) – operating system environment for running the forensic tools, since Steghide isn't available through Homebrew on macOS.

Docker – used to run a Kali Linux container on my Mac, with my assignment folder mounted directly into it so my work synced with my real project files.

Steghide – used to embed and extract hidden data inside the bitmap image (embed/extract commands, password-protected).

StegSeek – used to run a dictionary attack against the stego files to test password strength.

file – used to confirm the format and properties of the carrier image.

xxd – used to inspect the raw header bytes of the image files.

binutils (strings) – used to scan files for readable text content.

sha256sum / diff – used to generate and compare cryptographic hashes and file contents for integrity verification.

Git and GitHub—used for version control and to store and submit my evidence, commands, and report.

AI assistance and Google—used to help explain concepts (steganography vs. encryption, LSB substitution, hashing) in plain language and to break down and troubleshoot complex or compound terminal commands.

Grammarly: to review and improve the grammar and clarity of this report. All commands were run by me personally, and all results shown are from my own terminal output.

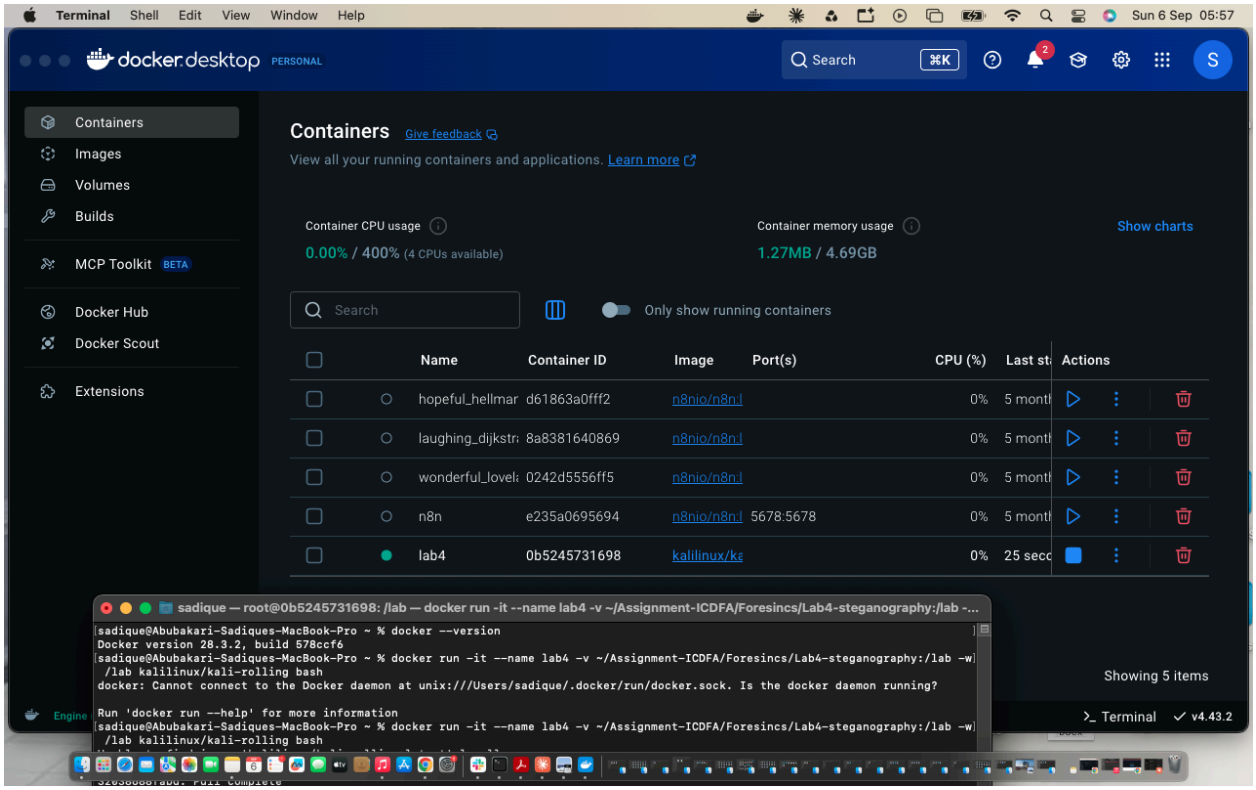
Introduction

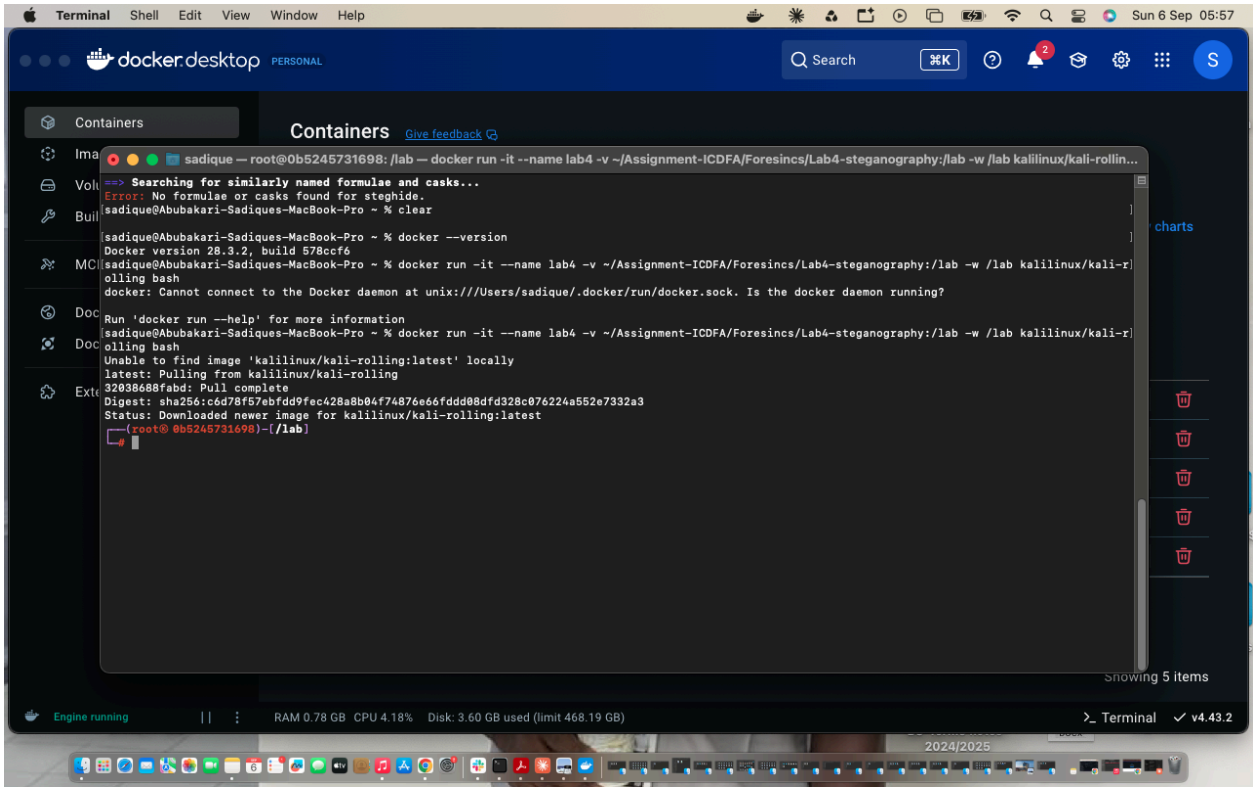
For this lab I looked into steganography—hiding a message inside a picture so that nobody can tell just by looking that anything is hidden there at all. I worked through hiding a text message inside a bitmap image, pulling it back out again, comparing the image before and after the message was hidden, and then trying to crack the password protecting it using a tool called StegSeek. After that, I repeated the whole process on my own, from scratch, with a different message and a different password, to prove to myself (and my instructor) that I actually understood what I was doing rather than just copying commands.

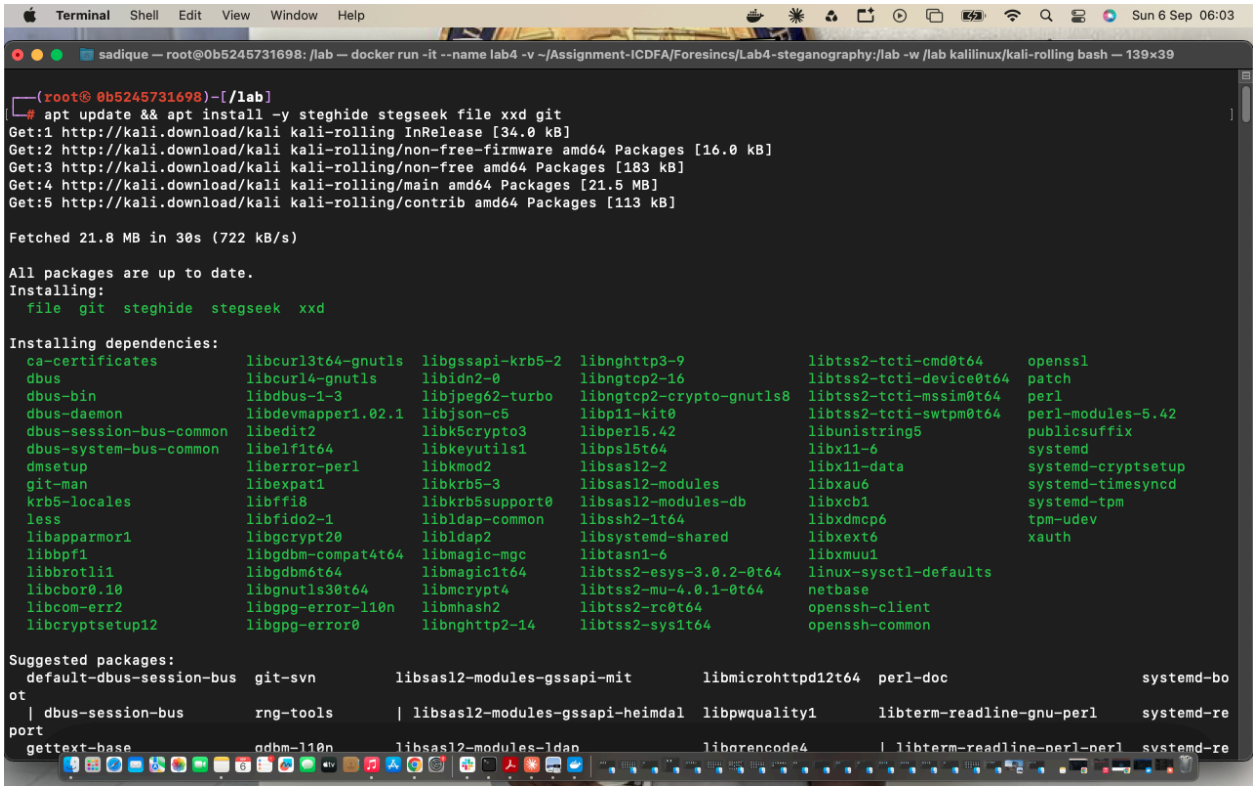
Setting Up

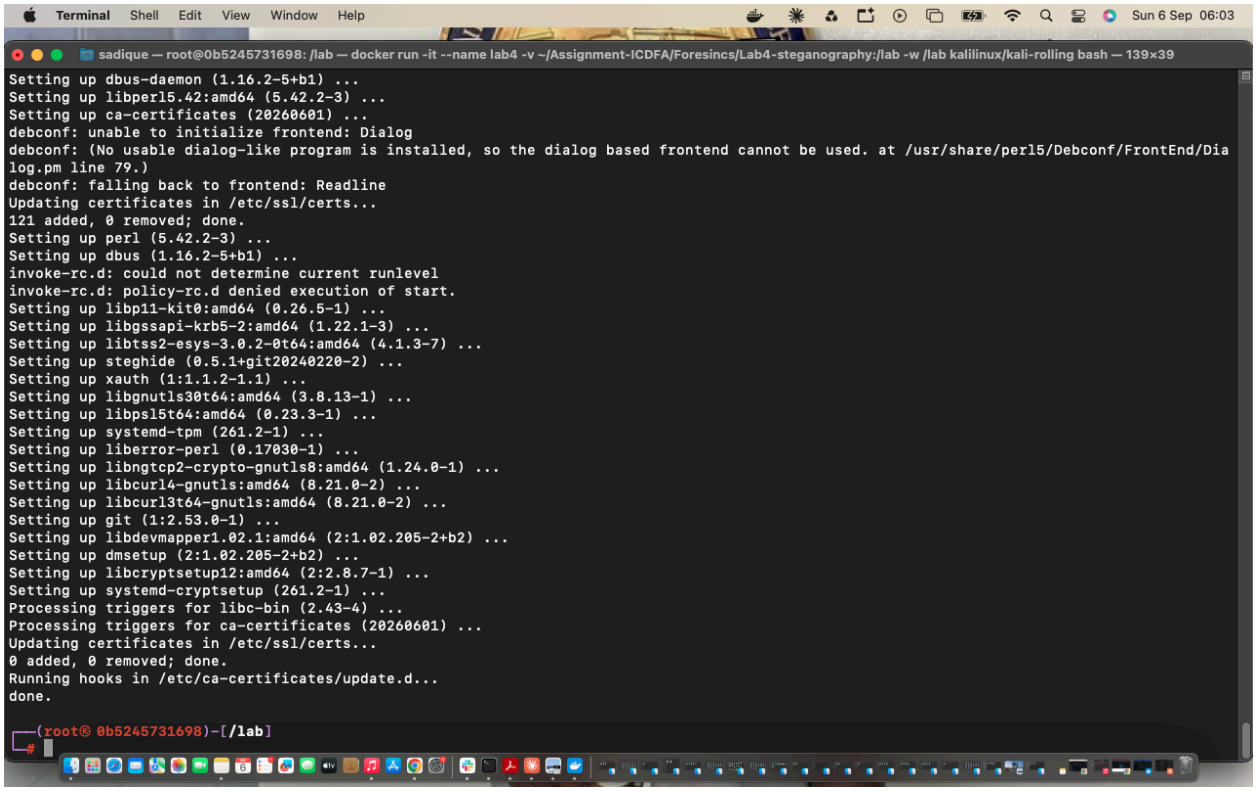
I did this on my MacBook, but the main tool for this lab, Steghide, isn't available through Homebrew anymore — it depends on an old library (libmcrypt) that Homebrew dropped years ago. Instead of fighting with that, I installed Docker and ran a Kali Linux container, which already has these forensic tools packaged for it. I mounted my actual assignment folder from my Mac straight into the container, so anything I created inside the container showed up directly in my real project folder and could be committed to GitHub normally from my Mac's terminal.

Inside the container, I installed the tools I needed: steghide (to hide and extract files), stegseek (to test passwords), file and binutils (to inspect files), and git (to commit and push my work).


Terminal Shell Edit View Window Help
Sun 6 Sep 05:57
docker desktop PERSONAL
Search ⌘K
Containers
Images
Volumes
Builds
MCP Toolkit BETA
Docker Hub
Docker Scout
Extensions
Containers
Give feedback
View all your running containers and applications. Learn more
Container CPU usage
0.00% / 400% (4 CPUs available)
Container memory usage
1.27MB / 4.69GB
Show charts
Search
Only show running containers
Name
Container ID
Image
Port(s)
CPU (%)
Actions
hopeful_hellmar d61863a0fff2 n8nio/n8n:l 0% 5 month
laughing_dijkstr 8a8381640869 n8nio/n8n:l 0% 5 month
wonderful_lovel 0242d5556ff5 n8nio/n8n:l 0% 5 month
n8n e235a0695694 n8nio/n8n:l 5678:5678 0% 5 month
lab4 0b5245731698 kalilinux/ka 0% 25 secc
sadique — root@0b5245731698: /lab — docker run -it --name lab4 -v ~/Assignment-ICDFA/Foresincs/Lab4-steganography:/lab -...
sadique@Abubakari-Sadiques-MacBook-Pro ~ % docker --version
Docker version 28.3.2, build 578ccf6
sadique@Abubakari-Sadiques-MacBook-Pro ~ % docker run -it --name lab4 -v ~/Assignment-ICDFA/Foresincs/Lab4-steganography:/lab -w
/lab kalilinux/kali-rolling bash
docker: Cannot connect to the Docker daemon at unix:///Users/sadique/.docker/run/docker.sock. Is the docker daemon running?
Run 'docker run --help' for more information
sadique@Abubakari-Sadiques-MacBook-Pro ~ % docker run -it --name lab4 -v ~/Assignment-ICDFA/Foresincs/Lab4-steganography:/lab -w
/lab kalilinux/kali-rolling bash
Showing 5 items
Engine
Terminal
v4.43.2


Terminal Shell Edit View Window Help
Sun 6 Sep 05:57
docker desktop PERSONAL
Search ⌘K
Containers
Containers
Give feedback
sadique — root@0b5245731698: /lab — docker run -it --name lab4 -v ~/Assignment-ICDFA/Foresincs/Lab4-steganography:/lab -w /lab kalilinux/kali-rollin...
==> Searching for similarly named formulae and casks...
Error: No formulae or casks found for steghide.
sadique@Abubakari-Sadiques-MacBook-Pro ~ % clear
sadique@Abubakari-Sadiques-MacBook-Pro ~ % docker --version
Docker version 28.3.2, build 578ccf6
sadique@Abubakari-Sadiques-MacBook-Pro ~ % docker run -it --name lab4 -v ~/Assignment-ICDFA/Foresincs/Lab4-steganography:/lab -w /lab kalilinux/kali-r
olling bash
docker: Cannot connect to the Docker daemon at unix:///Users/sadique/.docker/run/docker.sock. Is the docker daemon running?
Run 'docker run --help' for more information
sadique@Abubakari-Sadiques-MacBook-Pro ~ % docker run -it --name lab4 -v ~/Assignment-ICDFA/Foresincs/Lab4-steganography:/lab -w /lab kalilinux/kali-r
olling bash
Unable to find image 'kalilinux/kali-rolling:latest' locally
latest: Pulling from kalilinux/kali-rolling
32038688fabd: Pull complete
Digest: sha256:c6d78f57ebfdd9fec428a8b04f74876e66fddd08dfd328c076224a552e7332a3
Status: Downloaded newer image for kalilinux/kali-rolling:latest
(root㉿0b5245731698)-[/lab]
#
Showing 5 items
Engine running
RAM 0.78 GB CPU 4.18% Disk: 3.60 GB used (limit 468.19 GB)
Terminal
v4.43.2
2024/2025


Terminal Shell Edit View Window Help
Sun 6 Sep 06:03
sadique — root@0b5245731698: /lab — docker run -it --name lab4 -v ~/Assignment-ICDFA/Foresincs/Lab4-steganography:/lab -w /lab kalilinux/kali-rolling bash — 139×39
┌──(root㉿ 0b5245731698)-[/lab]
└─# apt update && apt install -y steghide stegseek file xxd git
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:3 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Packages [21.5 MB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Packages [113 kB]
Fetched 21.8 MB in 30s (722 kB/s)
All packages are up to date.
Installing:
file git steghide stegseek xxd
Installing dependencies:
ca-certificates libcurl3t64-gnutls libgssapi-krb5-2 libnghttp3-9 libtss2-tcti-cmd0t64 openssl
dbus libcurl4-gnutls libidn2-0 libngtcp2-16 libtss2-tcti-device0t64 patch
dbus-bin libdbus-1-3 libjpeg62-turbo libngtcp2-crypto-gnutls8 libtss2-tcti-mssim0t64 perl
dbus-daemon libdevmapper1.02.1 libjson-c5 libp11-kit0 libtss2-tcti-swtpm0t64 perl-modules-5.42
dbus-session-bus-common libedit2 libk5crypto3 libperl5.42 libunistring5 publicsuffix
dbus-system-bus-common libelf1t64 libkeyutils1 libpsl5t64 libx11-6 systemd
dmsetup liberror-perl libkmod2 libsasl2-2 libx11-data systemd-cryptsetup
git-man libexpat1 libkrb5-3 libsasl2-modules libxau6 systemd-timesyncd
krb5-locales libffi8 libkrb5support0 libsasl2-modules-db libxcb1 systemd-tpm
less libfido2-1 libldap-common libssh2-1t64 libxdmcp6 tpm-udev
libapparmor1 libgcrypt20 libldap2 libsystemd-shared libxext6 xauth
libbpf1 libgdbm-compat4t64 libmagic-mgc libtasn1-6 libxmuu1
libbrotli1 libgdbm6t64 libmagic1t64 libtss2-esys-3.0.2-0t64 linux-sysctl-defaults
libcbor0.10 libgnutls30t64 libmcrypt4 libtss2-mu-4.0.1-0t64 netbase
libcom-err2 libgpg-error-l10n libmhash2 libtss2-rc0t64 openssh-client
libcryptsetup12 libgpg-error0 libnghttp2-14 libtss2-sys1t64 openssh-common
Suggested packages:
default-dbus-session-bus git-svn libsasl2-modules-gssapi-mit libmicrohttpd12t64 perl-doc systemd-bo
ot
| dbus-session-bus rng-tools | libsasl2-modules-gssapi-heimdal libpwquality1 libterm-readline-gnu-perl systemd-re
port
gettext-base gdbm-l10n libsasl2-modules-ldap libgrencode4 | libterm-readline-perl-perl systemd-re

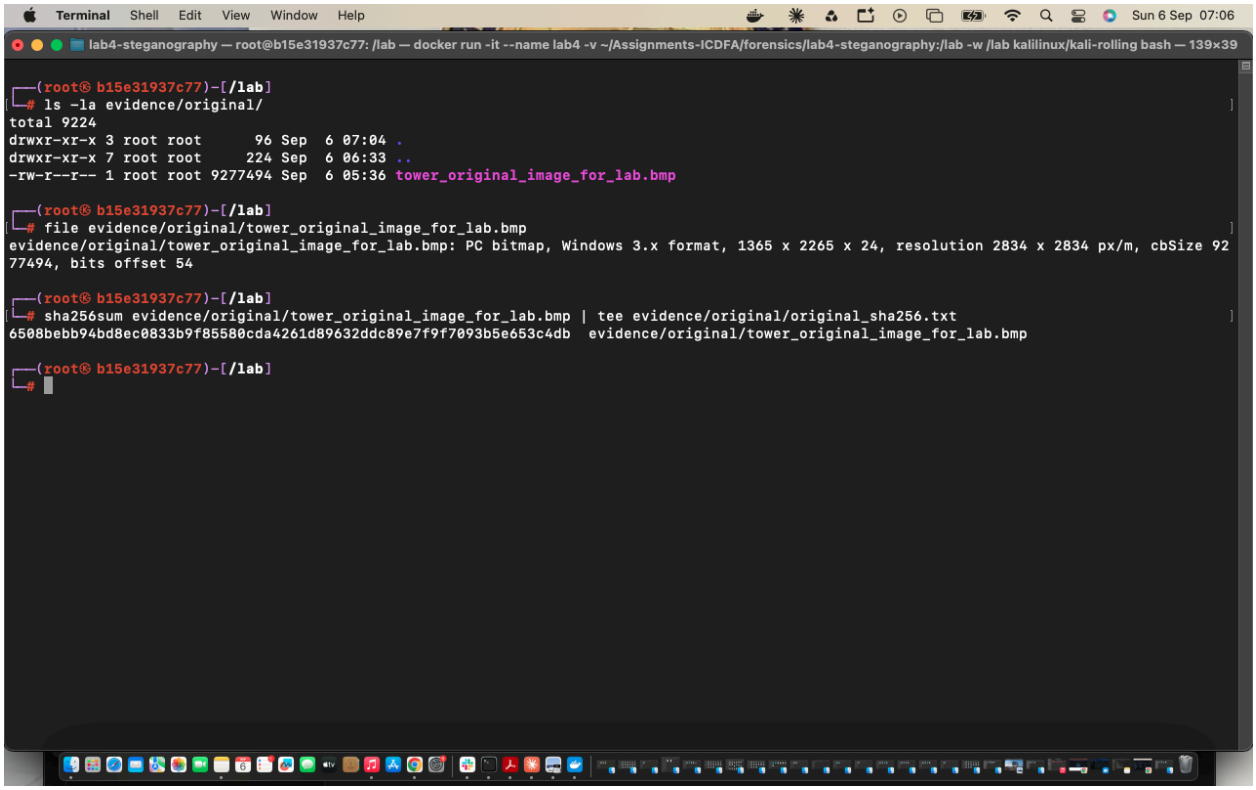

Terminal Shell Edit View Window Help
Sun 6 Sep 06:03
sadique — root@0b5245731698: /lab — docker run -it --name lab4 -v ~/Assignment-ICDFA/Foresincs/Lab4-steganography:/lab -w /lab kalilinux/kali-rolling bash — 139×39
Setting up dbus-daemon (1.16.2-5+b1) ...
Setting up libperl5.42:amd64 (5.42.2-3) ...
Setting up ca-certificates (20260601) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dia
log.pm line 79.)
debconf: falling back to frontend: Readline
Updating certificates in /etc/ssl/certs...
121 added, 0 removed; done.
Setting up perl (5.42.2-3) ...
Setting up dbus (1.16.2-5+b1) ...
invoke-rc.d: could not determine current runlevel
invoke-rc.d: policy-rc.d denied execution of start.
Setting up libp11-kit0:amd64 (0.26.5-1) ...
Setting up libgssapi-krb5-2:amd64 (1.22.1-3) ...
Setting up libtss2-esys-3.0.2-0t64:amd64 (4.1.3-7) ...
Setting up steghide (0.5.1+git20240220-2) ...
Setting up xauth (1:1.1.2-1.1) ...
Setting up libgnutls30t64:amd64 (3.8.13-1) ...
Setting up libpsl5t64:amd64 (0.23.3-1) ...
Setting up systemd-tpm (261.2-1) ...
Setting up liberror-perl (0.17030-1) ...
Setting up libngtcp2-crypto-gnutls8:amd64 (1.24.0-1) ...
Setting up libcurl4-gnutls:amd64 (8.21.0-2) ...
Setting up libcurl3t64-gnutls:amd64 (8.21.0-2) ...
Setting up git (1:2.53.0-1) ...
Setting up libdevmapper1.02.1:amd64 (2:1.02.205-2+b2) ...
Setting up dmsetup (2:1.02.205-2+b2) ...
Setting up libcryptsetup12:amd64 (2:2.8.7-1) ...
Setting up systemd-cryptsetup (261.2-1) ...
Processing triggers for libc-bin (2.43-4) ...
Processing triggers for ca-certificates (20260601) ...
Updating certificates in /etc/ssl/certs...
0 added, 0 removed; done.
Running hooks in /etc/ca-certificates/update.d...
done.
┌──(root㉿ 0b5245731698)-[/lab]
└─#

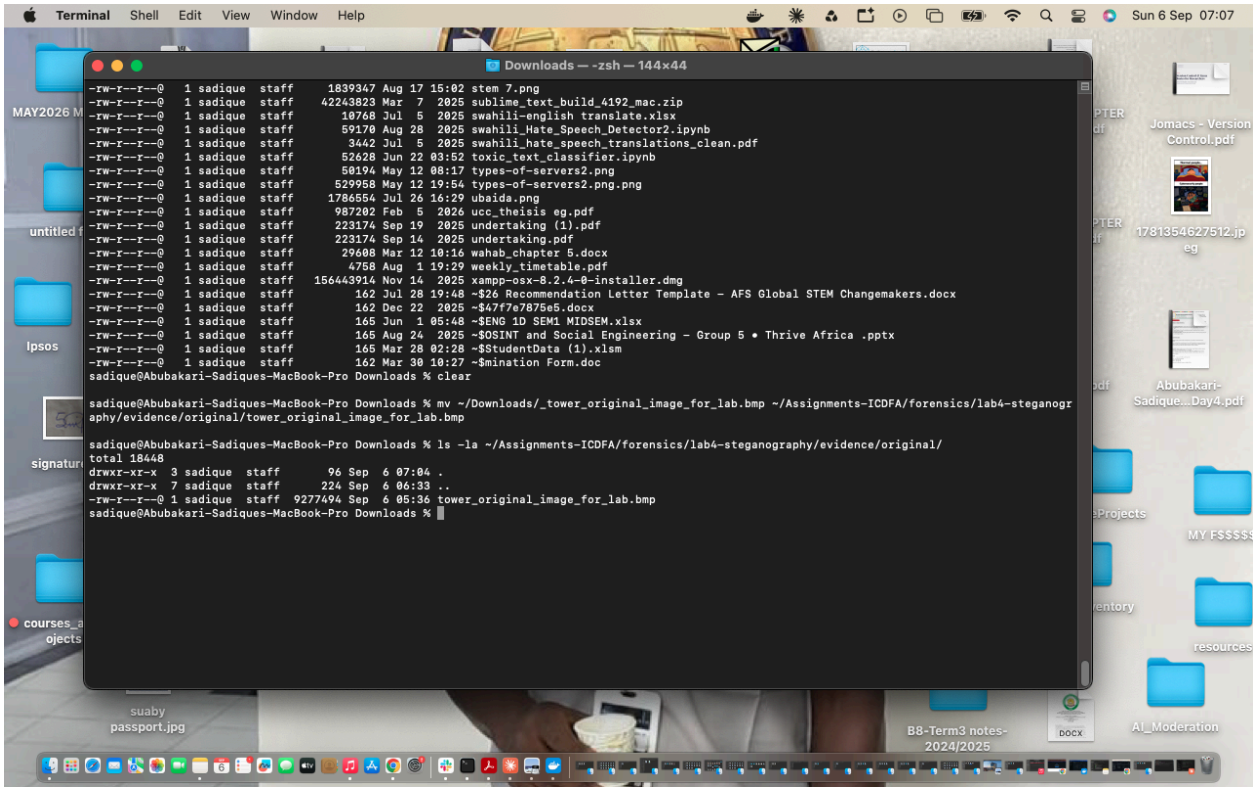
Part A – Getting the Carrier Image Ready

Before hiding anything, I needed a “carrier” image; the picture that would end up holding the secret message. I used a bitmap (.bmp) photo of a tower. Bitmap images work well for this kind of exercise because of how they store their picture data (more on that in Part C). I downloaded the image, used the file command to confirm what kind of file it actually was, and then ran it through sha256sum to generate a fingerprint (a hash) of it. That fingerprint matters because if even a single bit of the file changes later on, the hash comes out completely different; so it's proof of exactly what the file looked like at this point, before I touched it.

What I ran and what happened

```
$ file tower_original_image_for_lab.bmp
tower_original_image_for_lab.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24

$ sha256sum tower_original_image_for_lab.bmp
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db
tower_original_image_for_lab.bmp
```





Part B – Creating the Secret File

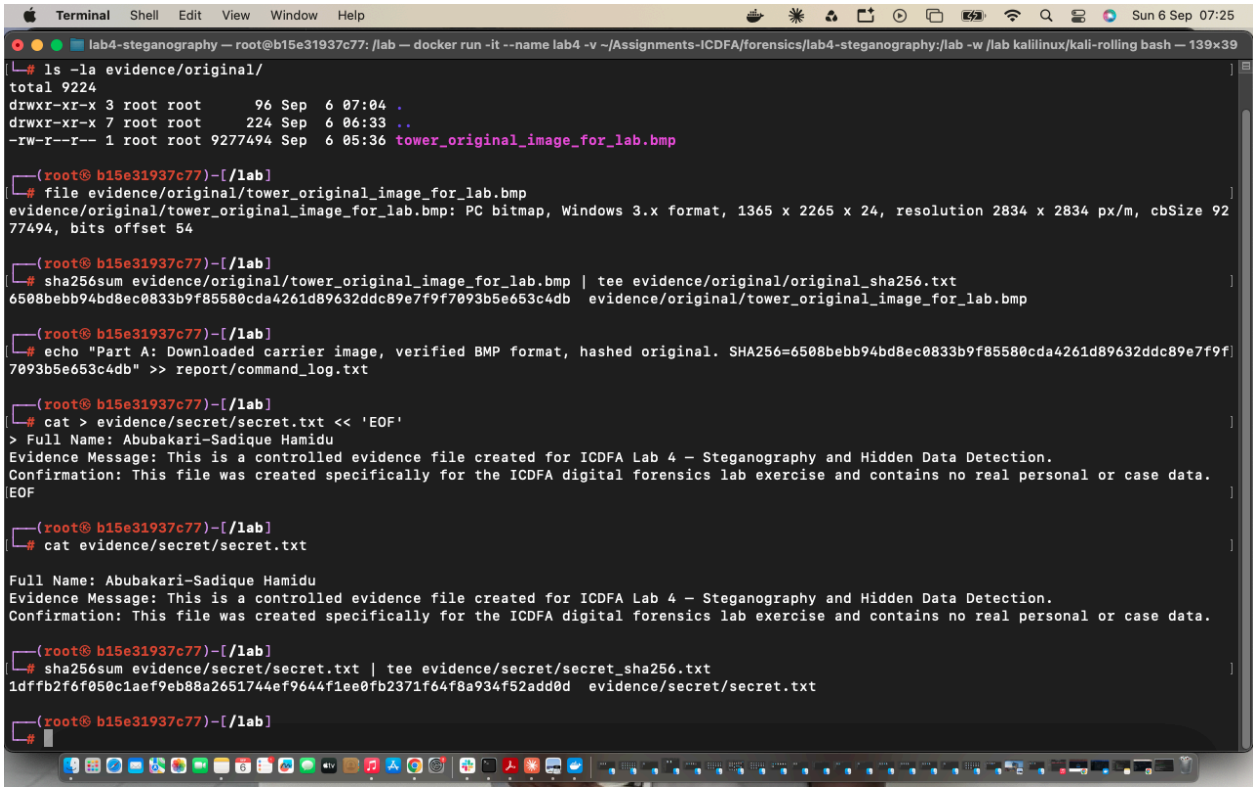
Next I made the actual secret message I'd be hiding a small text file with my name and a short evidence statement, so it was clearly tied to me and to this specific lab. I hashed this file too, for the same reason as the image: so I'd have proof of exactly what it said before it got hidden and pulled back out again.

Contents of secret.txt

> *Full Name: Abubakari-Sadique Hamidu*
> *Evidence Message: This is a controlled evidence file created for ICDFA Lab 4 — Steganography and Hidden Data Detection.*
> *Confirmation: This file was created specifically for the ICDFA digital forensics lab exercise and contains no real personal or case data.*

What I ran and what happened

```
$ sha256sum secret.txt
1dffb2f6f050c1aef9eb88a2651744ef9644f1ee0fb2371f64f8a934f52add0d  secret.txt
```



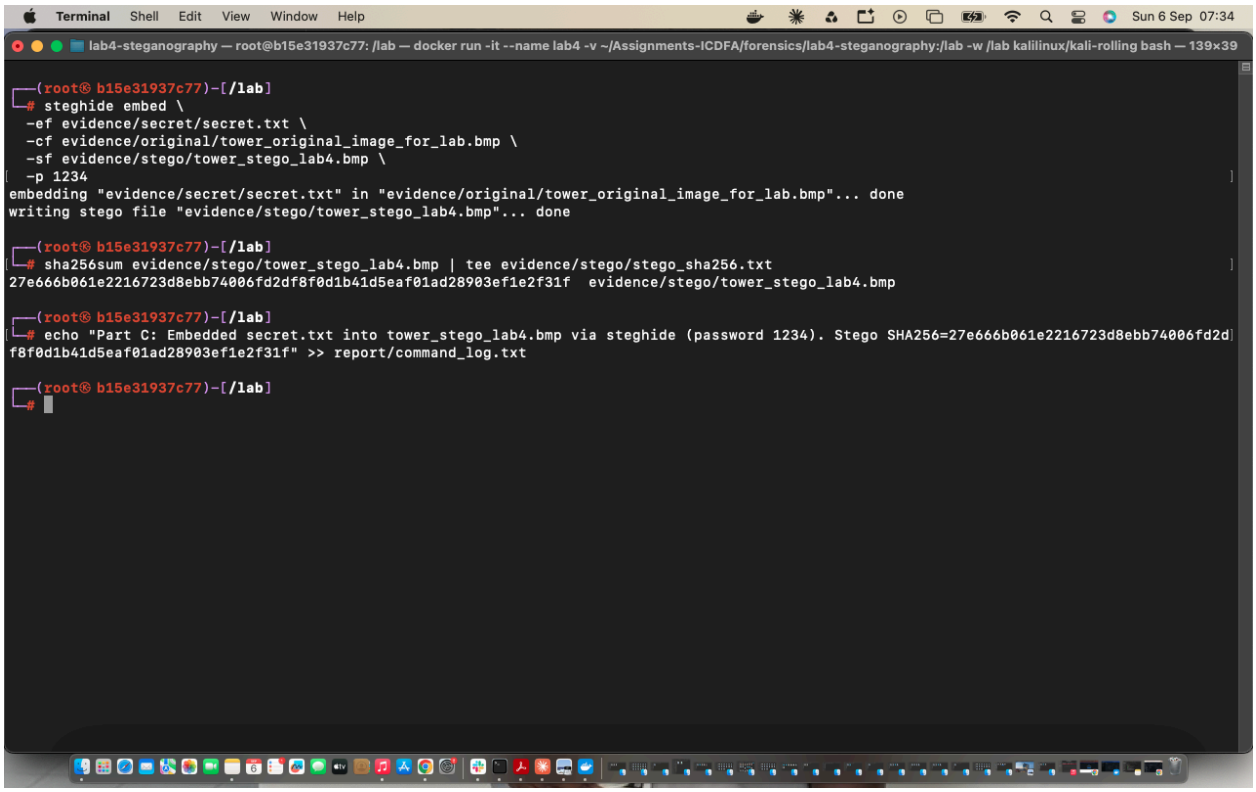
Part C – Hiding the Message

This is where the actual steganography happens. I used Steghide to embed secret.txt inside the bitmap image, protected with the password 1234. Steghide works by quietly changing the least important bit of some of the pixel color values in the image, changes so small they don't visibly affect the picture at all, but line up to spell out the hidden message once you know the password. It also encrypts the message with that password before hiding it, so even if someone figured out the technique, they'd still need the password to actually read what's inside.

What I ran and what happened

```
$ steghide embed -ef secret.txt -cf tower_original_image_for_lab.bmp -sf
tower_stego_lab4.bmp -p 1234
embedding "secret.txt" in "tower_original_image_for_lab.bmp"... done
writing stego file "tower_stego_lab4.bmp"... done

$ sha256sum tower_stego_lab4.bmp
27e666b061e2216723d8ebb74006fd2df8f0d1b41d5eaf01ad28903ef1e2f31f tower_stego_lab4.bmp
```

Notice the hash is completely different from the original image's hash, even though nothing about the picture looks any different to the eye. That's the whole point; the file's content changed, but not in any way a person would notice just by looking at it.



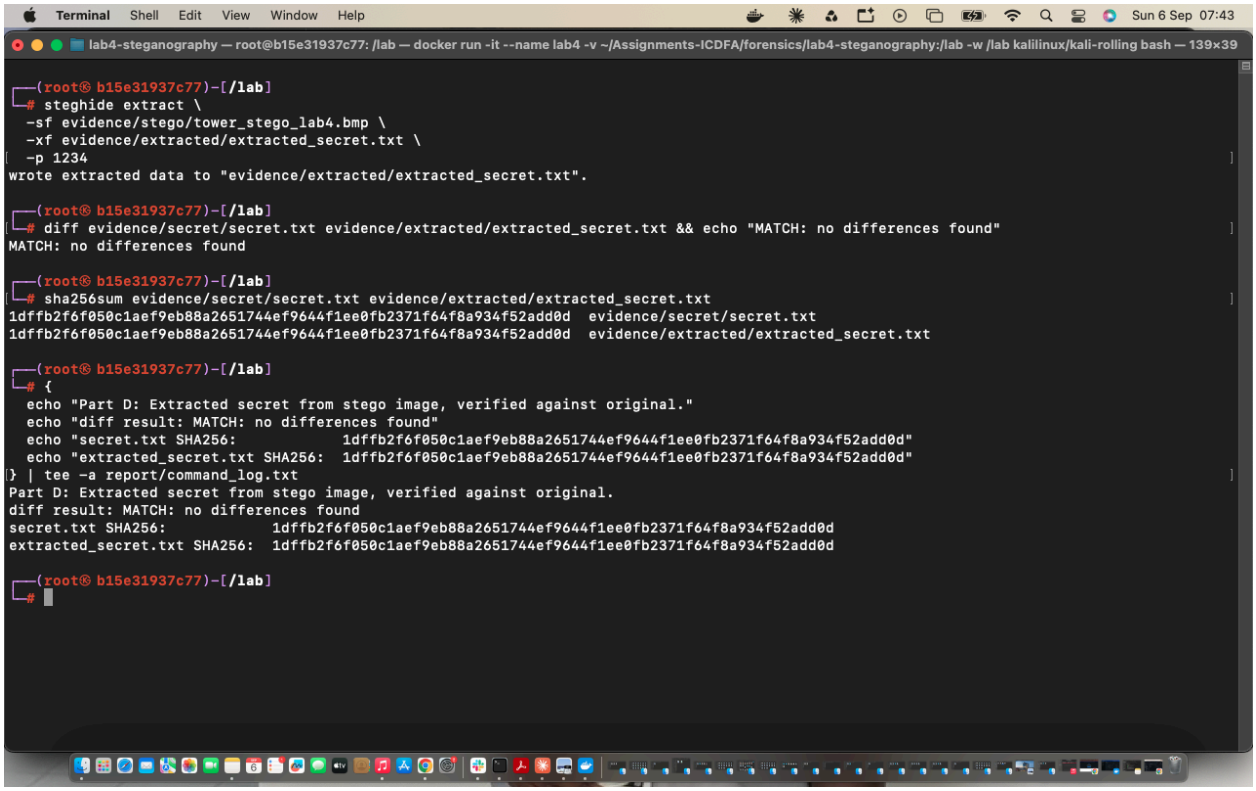
Part D – Getting the Message Back Out

To prove the hiding actually worked (and that nothing got corrupted along the way), I extracted the message back out of the stego image using the same password, then checked it against the original secret.txt two different ways: diff, which compares text content line by line, and sha256sum again, which checks it's identical all the way down to the byte.

What I ran and what happened

```
$ steghide extract -sf tower_stego_lab4.bmp -xf extracted_secret.txt -p 1234
wrote extracted data to "extracted_secret.txt".

$ diff secret.txt extracted_secret.txt
(no output at all – meaning no differences were found)

$ sha256sum extracted_secret.txt
1dffb2f6f050c1aef9eb88a2651744ef9644f1ee0fb2371f64f8a934f52add0d  extracted_secret.txt
```

Both hashes match exactly (1dffb2f6...add0d), and diff came back silent, which means the message came back out exactly the way it went in.



Part E – Comparing the Original and Stego Images

Now I wanted to look at this from the outside, the way someone examining evidence would: what actually changed between the original picture and the one with a message hidden inside it? I checked four things — the file size, the hash, the file's header bytes (the part right at the start of the file that describes what kind of image it is), and whether any of my secret text could be spotted just by scanning the file for plain readable text.

File size

```
$ ls -l tower_original_image_for_lab.bmp tower_stego_lab4.bmp
Both files: 9,277,494 bytes — exactly the same size.
```

The sizes match exactly. That makes sense once you know how this hiding technique works: it swaps existing bits inside the image rather than adding new data, so the file doesn't grow at all.

Hash

```
$ sha256sum tower_original_image_for_lab.bmp tower_stego_lab4.bmp
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db
tower_original_image_for_lab.bmp
27e666b061e2216723d8ebb74006fd2df8f0d1b41d5eaf01ad28903ef1e2f31f  tower_stego_lab4.bmp
```

The hashes are completely different, which proves the content did change even though the size didn't.

Header bytes

```
$ xxd -l 64 tower_original_image_for_lab.bmp
$ xxd -l 64 tower_stego_lab4.bmp
(the first 64 bytes of both files come back identical)
```

The header — the part of the file that tells a program “this is a bitmap image, this size, this format” — is untouched. Only the actual picture data further into the file changed.

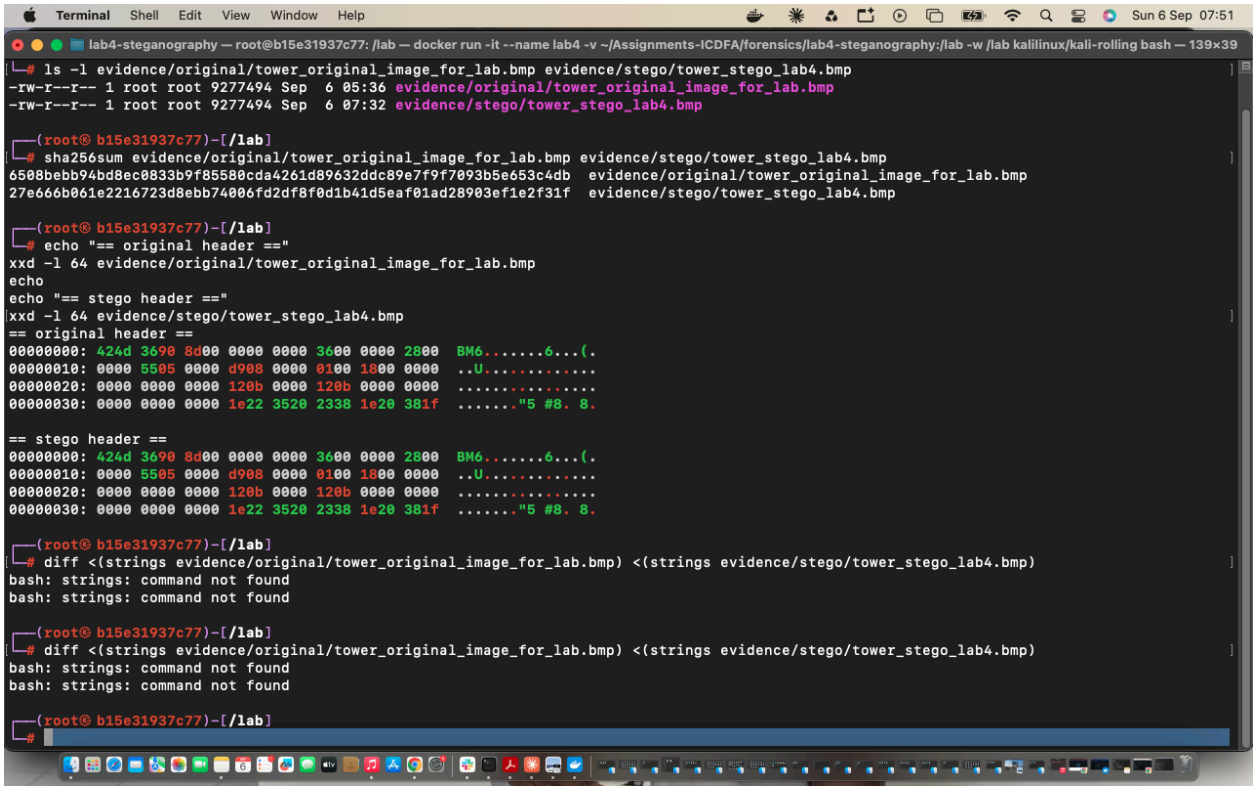
Readable text scan

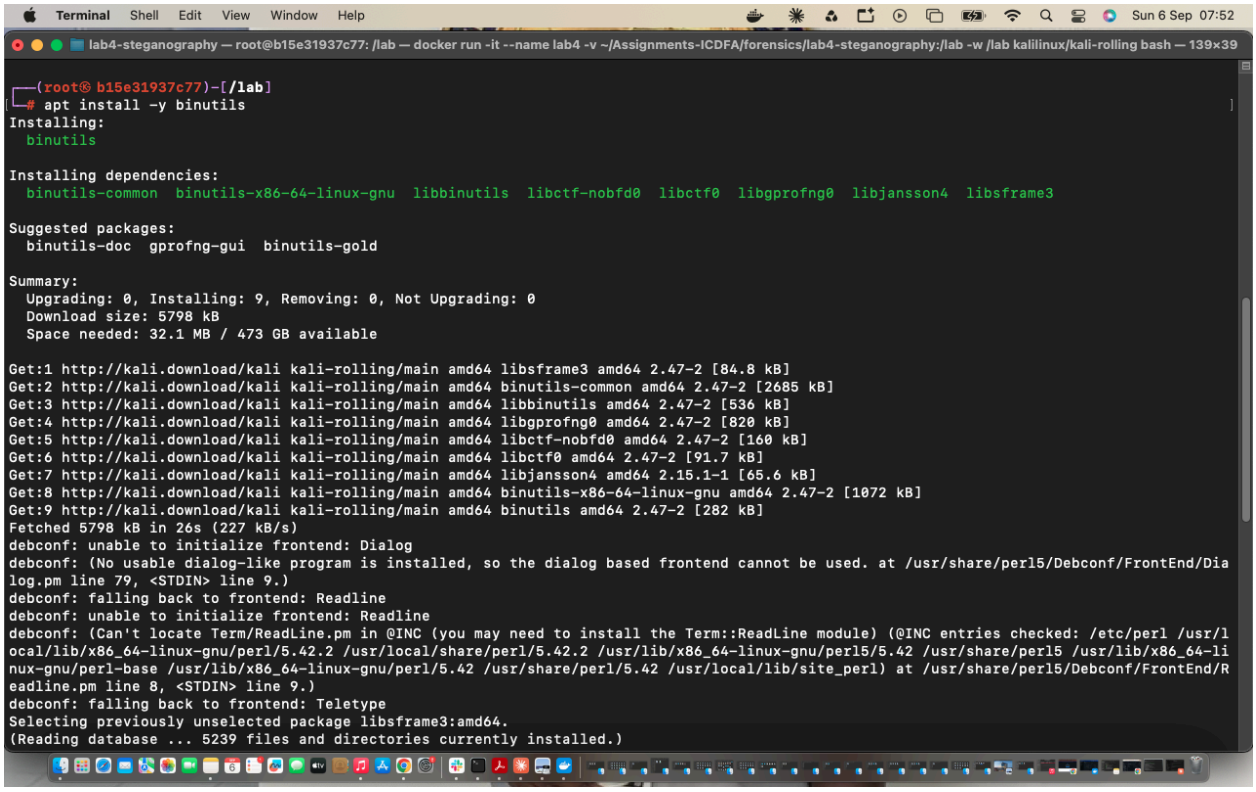
```
$ diff <(strings tower_original_image_for_lab.bmp) <(strings tower_stego_lab4.bmp) | wc
-l
2428

$ strings tower_stego_lab4.bmp | grep -i "Abubakari"
(nothing came back)
```

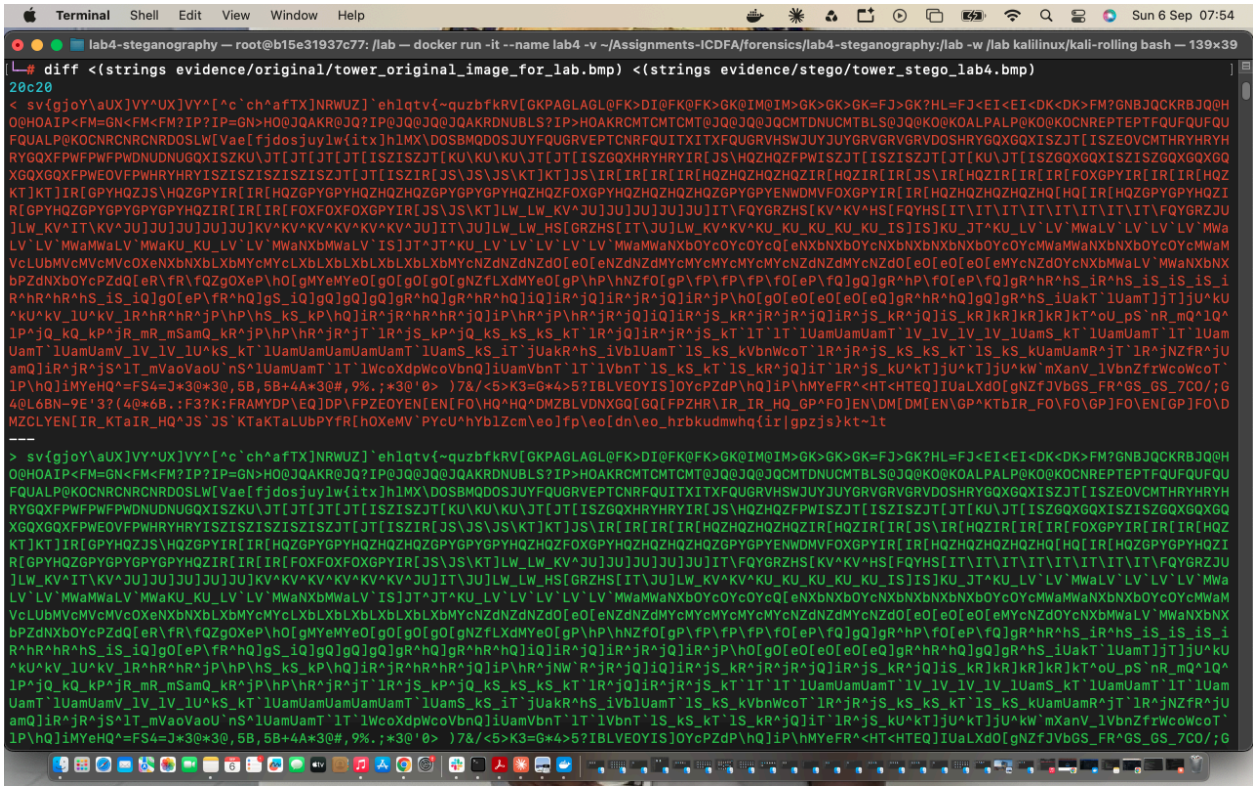
So 2,428 lines worth of differences turned up between the two files at the byte level, plenty changed. But even after scanning the entire stego file for any plain readable text, my name and message never show up, because Steghide encrypts the message before hiding it. Someone just glancing at this image, or even running a basic text scan on it, would never suspect anything was hidden inside.

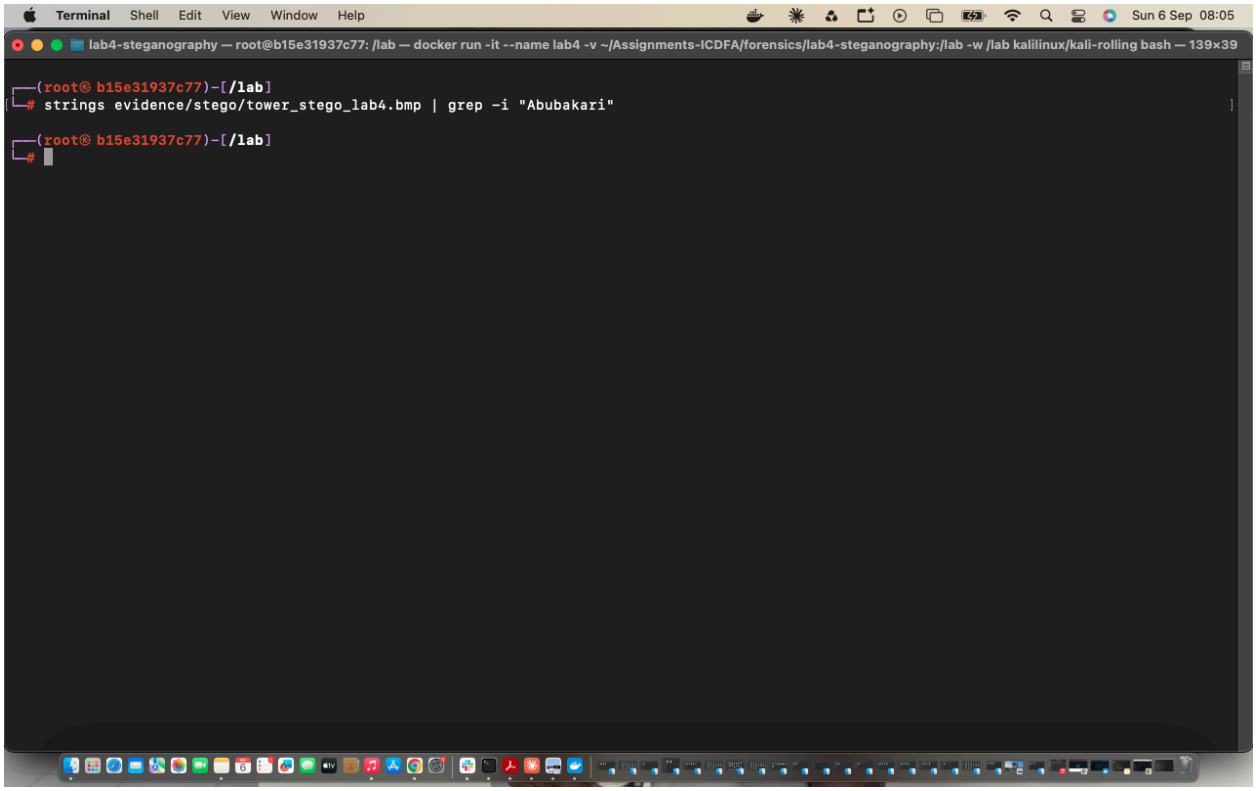
What this told me overall: this style of steganography (called LSB substitution, because it substitutes the least significant bit of certain bytes) keeps the file size and header exactly the same. Nothing about the

file “looks” different from the outside. All the real changes happen deep in the pixel data, and combined with encryption, the hidden content stays invisible to normal inspection.


Terminal Shell Edit View Window Help
Sun 6 Sep 07:51
lab4-steganography — root@b15e31937c77: /lab — docker run -it --name lab4 -v ~/Assignments-ICDFA/forensics/lab4-steganography:/lab -w /lab kalilinux/kali-rolling bash — 139×39
ls -l evidence/original/tower_original_image_for_lab.bmp evidence/stego/tower_stego_lab4.bmp
-rw-r--r-- 1 root root 9277494 Sep 6 05:36 evidence/original/tower_original_image_for_lab.bmp
-rw-r--r-- 1 root root 9277494 Sep 6 07:32 evidence/stego/tower_stego_lab4.bmp
(root® b15e31937c77)-[/lab]
sha256sum evidence/original/tower_original_image_for_lab.bmp evidence/stego/tower_stego_lab4.bmp
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db evidence/original/tower_original_image_for_lab.bmp
27e666b061e2216723d8ebb74006fd2df8f0d1b41d5eaf01ad28903ef1e2f31f evidence/stego/tower_stego_lab4.bmp
(root® b15e31937c77)-[/lab]
echo "== original header =="
xxd -l 64 evidence/original/tower_original_image_for_lab.bmp
echo
echo "== stego header =="
xxd -l 64 evidence/stego/tower_stego_lab4.bmp
== original header ==
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800 BM6.......6...(.
00000010: 0000 5505 0000 d908 0000 0100 1800 0000 ..U.............
00000020: 0000 0000 0000 120b 0000 120b 0000 0000 ....
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f ...."5 #8. 8.
== stego header ==
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800 BM6.......6...(.
00000010: 0000 5505 0000 d908 0000 0100 1800 0000 ..U.............
00000020: 0000 0000 0000 120b 0000 120b 0000 0000 ....
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f ...."5 #8. 8.
(root® b15e31937c77)-[/lab]
diff <(strings evidence/original/tower_original_image_for_lab.bmp) <(strings evidence/stego/tower_stego_lab4.bmp)
bash: strings: command not found
bash: strings: command not found
(root® b15e31937c77)-[/lab]
diff <(strings evidence/original/tower_original_image_for_lab.bmp) <(strings evidence/stego/tower_stego_lab4.bmp)
bash: strings: command not found
bash: strings: command not found
(root® b15e31937c77)-[/lab]
#


Terminal Shell Edit View Window Help
Sun 6 Sep 07:52
lab4-steganography — root@b15e31937c77: /lab — docker run -it --name lab4 -v ~/Assignments-ICDFA/forensics/lab4-steganography:/lab -w /lab kalilinux/kali-rolling bash — 139×39
(root® b15e31937c77)-[/lab]
apt install -y binutils
Installing:
binutils
Installing dependencies:
binutils-common binutils-x86-64-linux-gnu libbinutils libctf-nobfd0 libctf0 libgprofng0 libjansson4 libsframe3
Suggested packages:
binutils-doc gprofng-gui binutils-gold
Summary:
Upgrading: 0, Installing: 9, Removing: 0, Not Upgrading: 0
Download size: 5798 kB
Space needed: 32.1 MB / 473 GB available
Get:1 http://kali.download/kali kali-rolling/main amd64 libsframe3 amd64 2.47-2 [84.8 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 binutils-common amd64 2.47-2 [2685 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 libbinutils amd64 2.47-2 [536 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 libgprofng0 amd64 2.47-2 [820 kB]
Get:5 http://kali.download/kali kali-rolling/main amd64 libctf-nobfd0 amd64 2.47-2 [160 kB]
Get:6 http://kali.download/kali kali-rolling/main amd64 libctf0 amd64 2.47-2 [91.7 kB]
Get:7 http://kali.download/kali kali-rolling/main amd64 libjansson4 amd64 2.15.1-1 [65.6 kB]
Get:8 http://kali.download/kali kali-rolling/main amd64 binutils-x86-64-linux-gnu amd64 2.47-2 [1072 kB]
Get:9 http://kali.download/kali kali-rolling/main amd64 binutils amd64 2.47-2 [282 kB]
Fetched 5798 kB in 26s (227 kB/s)
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 79, <STDIN> line 9.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (Can't locate Term/ReadLine.pm in @INC (you may need to install the Term::ReadLine module) (@INC entries checked: /etc/perl /usr/local/lib/x86_64-linux-gnu/perl/5.42.2 /usr/local/share/perl/5.42.2 /usr/lib/x86_64-linux-gnu/perl5/5.42 /usr/share/perl5 /usr/lib/x86_64-linux-gnu/perl-base /usr/lib/x86_64-linux-gnu/perl/5.42 /usr/share/perl/5.42 /usr/local/lib/site_perl) at /usr/share/perl5/Debconf/FrontEnd/Readline.pm line 8, <STDIN> line 9.)
debconf: falling back to frontend: Teletype
Selecting previously unselected package libsframe3:amd64.
(Reading database ... 5239 files and directories currently installed.)


Terminal Shell Edit View Window Help
Sun 6 Sep 07:54
lab4-steganography — root@b15e31937c77: /lab — docker run -it --name lab4 -v ~/Assignments-ICDFA/forensics/lab4-steganography:/lab -w /lab kalilinux/kali-rolling bash — 139×39
┌──(root㉿b15e31937c77)-[/lab]
└─# diff <(strings evidence/original/tower_original_image_for_lab.bmp) <(strings evidence/stego/tower_stego_lab4.bmp)
20c20
< sv{gjoY\aUX]VY^UX]VY^[^c`ch^afTX]NRWUZ]`ehlqtv{~quzbfkRV[GKPAGLAGL@FK>DI@FK@FK>GK@IM@IM>GK>GK>GK=FJ>GK?HL=FJ<EI<EI<DK<DK>FM?GNBJQCKRBJQ@HO@HOAIP<FM=GN<FM<FM?IP?IP=GN>HO@JQAKR@JQ?IP@JQ@JQ@JQAKRDNUBLS?IP>HOAKRCMTCMTCMT@JQ@JQ@JQCMTDNUCMTBLS@JQ@KO@KOALPALP@KQ@KOCNREPTEPTFQUFQUFQUFQUALP@KOCNRCNRCNRDOSLW[Vae[fjdosjuylw{itx]hlMX\DOSBMQDOSJUYFQUGRVEPTCNRFQUITXITXFQUGRVHSWJUYJUYGRVGRVGRVDOSHRYGQXGQXISZJT[ISZEOVCMTHRYHRYHRYGQXFPWFPWFPWDNUDNUGQXISZKU\JT[JT[JT[JT[ISZISZJT[KU\KU\KU\JT[JT[ISZGQXHRYHRYIR[JS\HQZHQZFPWISZJT[ISZISZJT[JT[KU\JT[ISZGQXGQXISZISZGQXGQXGQXGQXGQXFPWEOVFPWHRYHRYISZISZISZISZISZJT[JT[ISZIR[JS\JS\JS\KT]KT]JS\IR[IR[IR[IR[HQZHQZHQZHQZIR[HQZIR[IR[JS\IR[HQZIR[IR[IR[FOXGPYIR[IR[IR[HQZKT]KT]IR[GPYHQZJS\HQZGPYIR[IR[HQZGPYGPYHQZHQZHQZGPYGPYGPYHQZHQZFOXGPYHQZHQZHQZGPYGPYENWDMVFOXGPYIR[IR[HQZHQZHQZHQZHQ[HQ[IR[HQZGPYGPYHQZIR[GPYHQZGPYGPYGPYHQZIR[IR[IR[FOXFOXFOXGPYIR[JS\JS\KT]LW_LW_KV^JU]JU]JU]JU]IT\FQYGRZHS[KV^KV^HS[FQYHS[IT\IT\IT\IT\IT\IT\IT\FQYGRZJU]LW_KV^IT\KV^JU]JU]JU]JU]JU]KV^KV^KV^KV^KV^KV^JU]IT\JU]LW_LW_HS[GRZHS[IT\JU]LW_KV^KV^KU_KU_KU_KU_IS]IS]KU_JT^KU_LV`LV`MWaLV`LV`LV`LV`MWaLV`LV`MWaMWaLV`MWaKU_KU_LV`LV`MWaNXbMWaLV`IS]JT^JT^KU_LV`LV`LV`LV`LV`MWaMWaNXbOYcOYcOYcQ[eNXbNXbOYcNXbNXbNXbNXbOYcOYcMWaMWaNXbNXbOYcOYcMWaMVcLUbMVcMVcMVcOXeNXbNXbLXbMYcMYcLXbLXbLXbLXbLXbMYcNZdNZdNZdO[eO[eNZdNZdMYcMYcMYcMYcNZdNZdMYcNZdO[eO[eO[eO[eMYcNZdOYcNXbMWaLV`MWaNXbNXbPZdNXbOYcPZdQ[eR\fR\fQZgOXeP\hO[gMYeMYeO[gO[gO[gO[gNZfLXdMYeO[gP\hP\hNZfO[gP\fP\fP\fP\fO[eP\fQ]gQ]gR^hP\fO[eP\fQ]gR^hR^hS_iR^hS_iS_iS_iS_iR^hR^hR^hS_iS_iQ]gO[eP\fR^hQ]gS_iQ]gQ]gQ]gQ]gR^hQ]gR^hR^hQ]iQ]iR^jQ]iR^jR^jQ]iR^jP\hO[gO[eO[eO[eO[eQ]gR^hR^hQ]gQ]gR^hS_iUakT`lUamT]jT]jU^kU^kU^kV_lU^kV_lR^hR^hR^jP\hP\hS_kS_kP\hQ]iR^jR^hR^hR^jQ]iP\hR^jP\hR^jR^jQ]iQ]iR^jS_kR^jR^jR^jQ]iR^jS_kR^jQ]iS_kR]kR]kR]kR]kT^oU_pS`nR_mQ^lQ^lP^jQ_kQ_kP^jR_mR_mSamQ_kR^jP\hP\hR^jR^jT`lR^jS_kP^jQ_kS_kS_kS_kT`lR^jQ]iR^jR^jS_kT`lT`lT`lUamUamUamT`lV_lV_lV_lV_lUamS_kT`lUamUamT`lT`lUamUamT`lUamUamV_lV_lV_lU^kS_kT`lUamUamUamUamUamT`lUamS_kS_iT`jUakR^hS_iVblUamT`lS_kS_kVbnWcoT`lR^jR^jS_kS_kS_kT`lS_kS_kUamUamR^jT`lR^jNZfR^jUamQ]iR^jR^jS^lT_mVaoVaoU`nS^lUamUamT`lT`lWcoXdpWcoVbnQ]iUamVbnT`lT`lVbnT`lS_kS_kT`lS_kR^jQ]iT`lR^jS_kU^kT]jU^kT]jU^kW`mXanV_lVbnZfrWcoWcoT`lP\hQ]iMYeHQ^=FS4=J*3@*3@,5B,5B+4A*3@#,9%.;*3@'0> )7&/<5>K3=G*4>5?IBLVEOYIS]OYcPZdP\hQ]iP\hMYeFR^<HT<HTEQ]IUaLXdO[gNZfJVbGS_FR^GS_GS_7CO/;G4@L6BN-9E'3?(4@*6B.:F3?K:FRAMYDP\EQ]DP\FPZEOYEN[EN[FO\HQ^HQ^DMZBLVDNXGQ[GQ[FPZHR\IR_IR_HQ_GP^FO]EN\DM[DM[EN\GP^KTbIR_FO\FO\GP]FO\EN[GP]FO\DMZCLYEN[IR_KTaIR_HQ^JS`JS`KTaKTaLUbPYfR[hOXeMV`PYcU^hYblZcm\eo]fp\eo[dn\eo_hrbkudmwhq{ir|gpzjs}kt~lt
---
> sv{gjoY\aUX]VY^UX]VY^[^c`ch^afTX]NRWUZ]`ehlqtv{~quzbfkRV[GKPAGLAGL@FK>DI@FK@FK>GK@IM@IM>GK>GK>GK=FJ>GK?HL=FJ<EI<EI<DK<DK>FM?GNBJQCKRBJQ@HO@HOAIP<FM=GN<FM<FM?IP?IP=GN>HO@JQAKR@JQ?IP@JQ@JQ@JQAKRDNUBLS?IP>HOAKRCMTCMTCMT@JQ@JQ@JQCMTDNUCMTBLS@JQ@KO@KOALPALP@KQ@KOCNREPTEPTFQUFQUFQUFQUALP@KOCNRCNRCNRDOSLW[Vae[fjdosjuylw{itx]hlMX\DOSBMQDOSJUYFQUGRVEPTCNRFQUITXITXFQUGRVHSWJUYJUYGRVGRVGRVDOSHRYGQXGQXISZJT[ISZEOVCMTHRYHRYHRYGQXFPWFPWFPWDNUDNUGQXISZKU\JT[JT[JT[JT[ISZISZJT[KU\KU\KU\JT[JT[ISZGQXHRYHRYIR[JS\HQZHQZFPWISZJT[ISZISZJT[JT[KU\JT[ISZGQXGQXISZISZGQXGQXGQXGQXGQXFPWEOVFPWHRYHRYISZISZISZISZISZJT[JT[ISZIR[JS\JS\JS\KT]KT]JS\IR[IR[IR[IR[HQZHQZHQZHQZIR[HQZIR[IR[JS\IR[HQZIR[IR[IR[FOXGPYIR[IR[IR[HQZKT]KT]IR[GPYHQZJS\HQZGPYIR[IR[HQZGPYGPYHQZHQZHQZGPYGPYGPYHQZHQZFOXGPYHQZHQZHQZGPYGPYENWDMVFOXGPYIR[IR[HQZHQZHQZHQZHQ[HQ[IR[HQZGPYGPYHQZIR[GPYHQZGPYGPYGPYHQZIR[IR[IR[FOXFOXFOXGPYIR[JS\JS\KT]LW_LW_KV^JU]JU]JU]JU]IT\FQYGRZHS[KV^KV^HS[FQYHS[IT\IT\IT\IT\IT\IT\IT\FQYGRZJU]LW_KV^IT\KV^JU]JU]JU]JU]JU]KV^KV^KV^KV^KV^KV^JU]IT\JU]LW_LW_HS[GRZHS[IT\JU]LW_KV^KV^KU_KU_KU_KU_IS]IS]KU_JT^KU_LV`LV`MWaLV`LV`LV`LV`MWaLV`LV`MWaMWaLV`MWaKU_KU_LV`LV`MWaNXbMWaLV`IS]JT^JT^KU_LV`LV`LV`LV`LV`MWaMWaNXbOYcOYcOYcQ[eNXbNXbOYcNXbNXbNXbNXbOYcOYcMWaMWaNXbNXbOYcOYcMWaMVcLUbMVcMVcMVcOXeNXbNXbLXbMYcMYcLXbLXbLXbLXbLXbMYcNZdNZdNZdO[eO[eNZdNZdMYcMYcMYcMYcNZdNZdMYcNZdO[eO[eO[eO[eMYcNZdOYcNXbMWaLV`MWaNXbNXbPZdNXbOYcPZdQ[eR\fR\fQZgOXeP\hO[gMYeMYeO[gO[gO[gO[gNZfLXdMYeO[gP\hP\hNZfO[gP\fP\fP\fP\fO[eP\fQ]gQ]gR^hP\fO[eP\fQ]gR^hR^hS_iR^hS_iS_iS_iS_iR^hR^hR^hS_iS_iQ]gO[eP\fR^hQ]gS_iQ]gQ]gQ]gQ]gR^hQ]gR^hR^hQ]iQ]iR^jQ]iR^jR^jQ]iR^jP\hO[gO[eO[eO[eO[eQ]gR^hR^hQ]gQ]gR^hS_iUakT`lUamT]jT]jU^kU^kU^kV_lU^kV_lR^hR^hR^jP\hP\hS_kS_kP\hQ]iR^jR^hR^hR^jQ]iP\hR^jNW`R^jR^jQ]iQ]iR^jS_kR^jR^jR^jQ]iR^jS_kR^jQ]iS_kR]kR]kR]kR]kT^oU_pS`nR_mQ^lQ^lP^jQ_kQ_kP^jR_mR_mSamQ_kR^jP\hP\hR^jR^jT`lR^jS_kP^jQ_kS_kS_kS_kT`lR^jQ]iR^jR^jS_kT`lT`lT`lUamUamUamT`lV_lV_lV_lV_lUamS_kT`lUamUamT`lT`lUamUamT`lUamUamV_lV_lV_lU^kS_kT`lUamUamUamUamUamT`lUamS_kS_iT`jUakR^hS_iVblUamT`lS_kS_kVbnWcoT`lR^jR^jS_kS_kS_kT`lS_kS_kUamUamR^jT`lR^jNZfR^jUamQ]iR^jR^jS^lT_mVaoVaoU`nS^lUamUamT`lT`lWcoXdpWcoVbnQ]iUamVbnT`lT`lVbnT`lS_kS_kT`lS_kR^jQ]iT`lR^jS_kU^kT]jU^kT]jU^kW`mXanV_lVbnZfrWcoWcoT`lP\hQ]iMYeHQ^=FS4=J*3@*3@,5B,5B+4A*3@#,9%.;*3@'0>)7&/<5>K3=G*4>5?IBLVEOYIS]OYcPZdP\hQ]iP\hMYeFR^<HT<HTEQ]IUaLXdO[gNZfJVbGS_FR^GS_GS_7CO/;G


Terminal Shell Edit View Window Help
Sun 6 Sep 08:05
lab4-steganography — root@b15e31937c77: /lab — docker run -it --name lab4 -v ~/Assignments-ICDFA/forensics/lab4-steganography:/lab -w /lab kalilinux/kali-rolling bash — 139×39
┌──(root㉿b15e31937c77)-[/lab]
└─# strings evidence/stego/tower_stego_lab4.bmp | grep -i "Abubakari"
┌──(root㉿b15e31937c77)-[/lab]
└─#

Part F – Trying to Crack the Password

So the hiding technique holds up fine to casual inspection, but what about the password protecting it? I used StegSeek, a tool built specifically to guess Steghide passwords quickly by testing a list of candidate words instead of guessing randomly, one at a time, the slow way. I put together a small wordlist with a handful of common passwords, and mixed the real one, 1234, in among them.

Wordlist used

```
password
letmein
qwerty
1234
admin123
forensics
steganography
2026lab
```

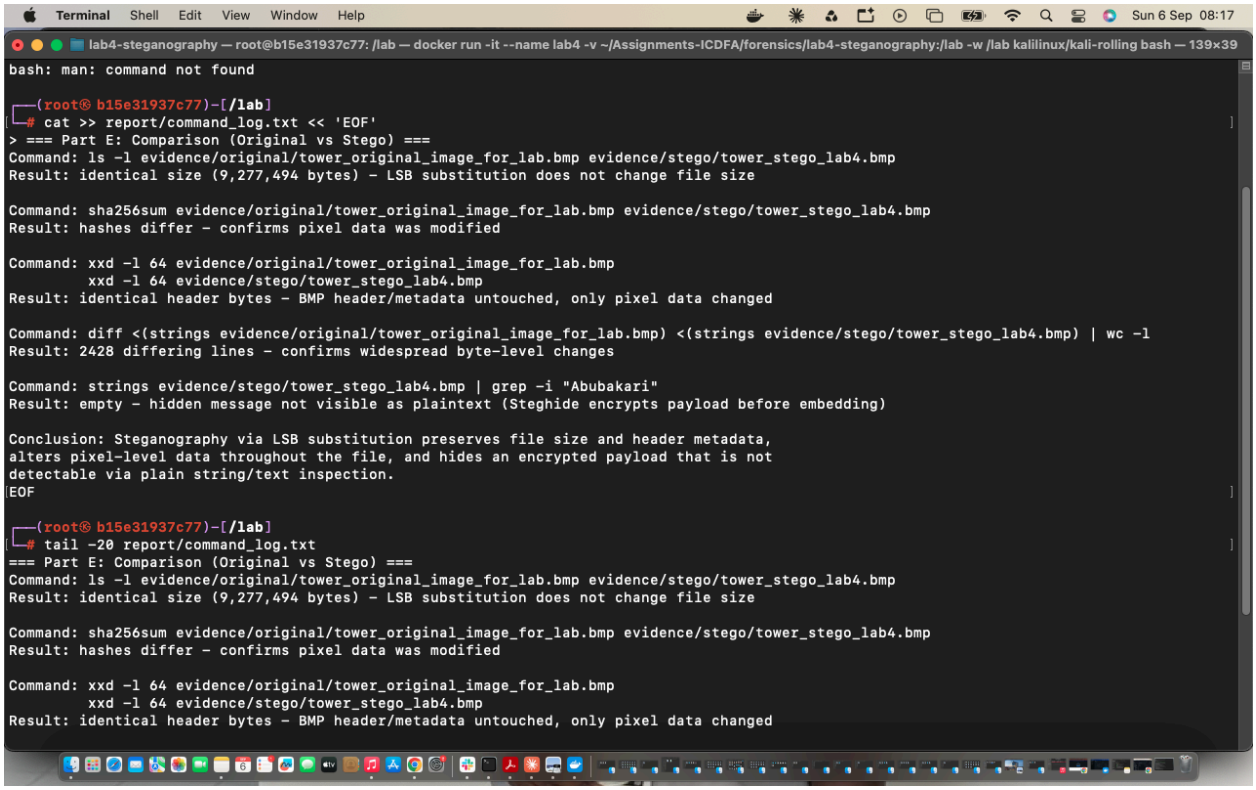
What I ran and what happened

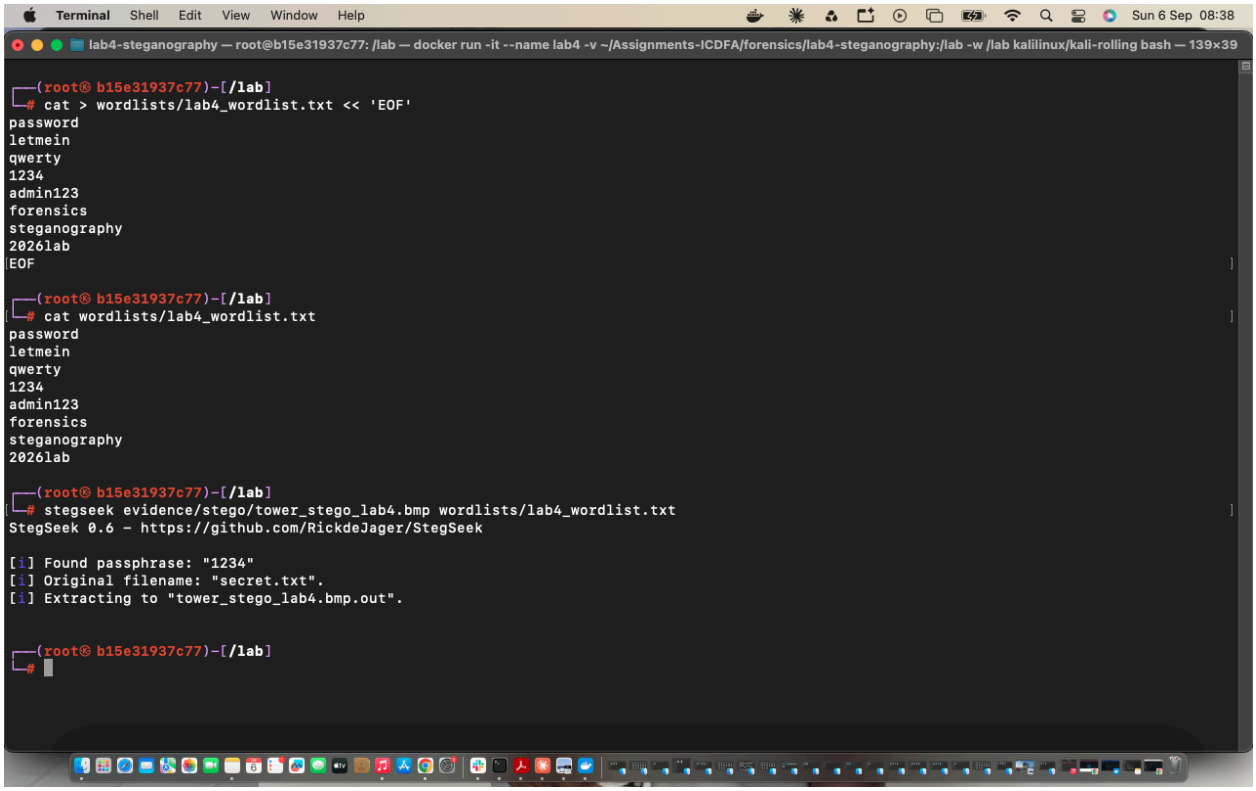
```
$ stegseek tower_stego_lab4.bmp lab4_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "1234"
[i] Original filename: "secret.txt".
[i] Extracting to "tower_stego_lab4.bmp.out".
```

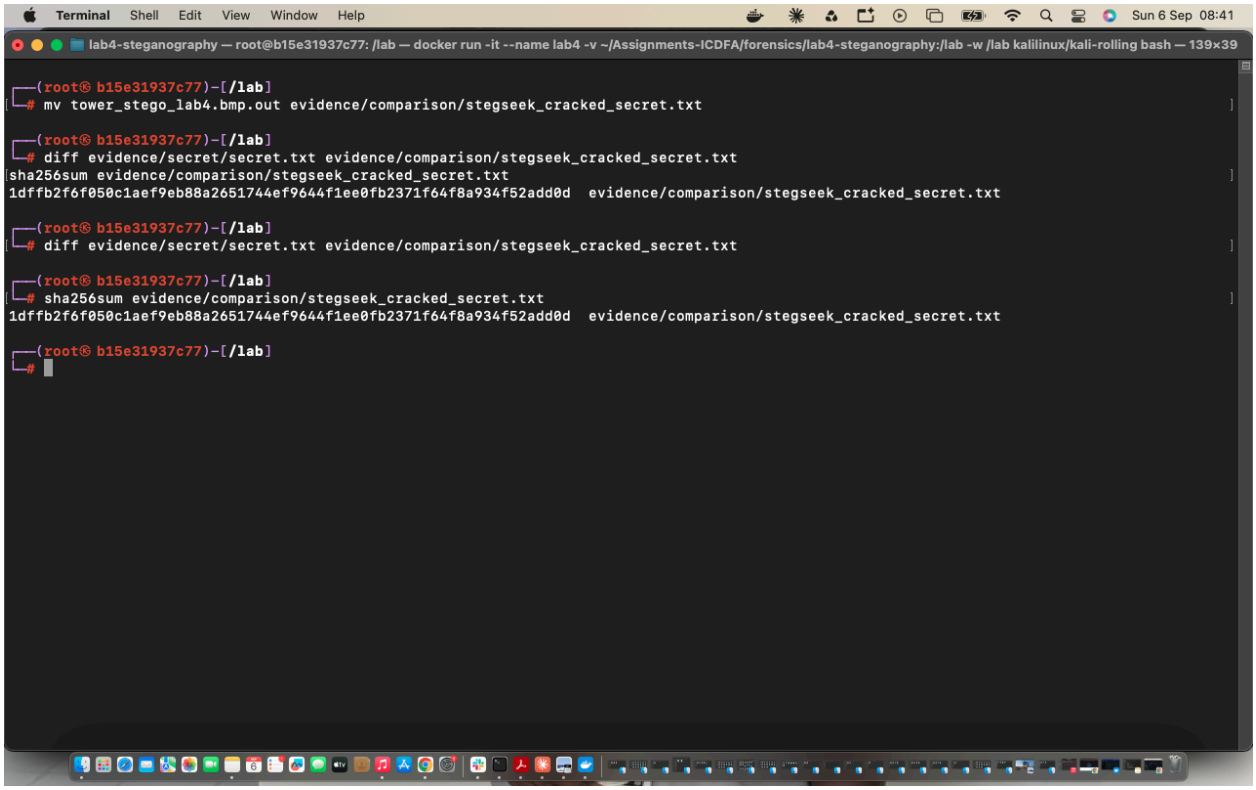
It found the password in seconds. I then checked the file it pulled out against my original secret.txt, to make sure it wasn't just a lucky filename match:

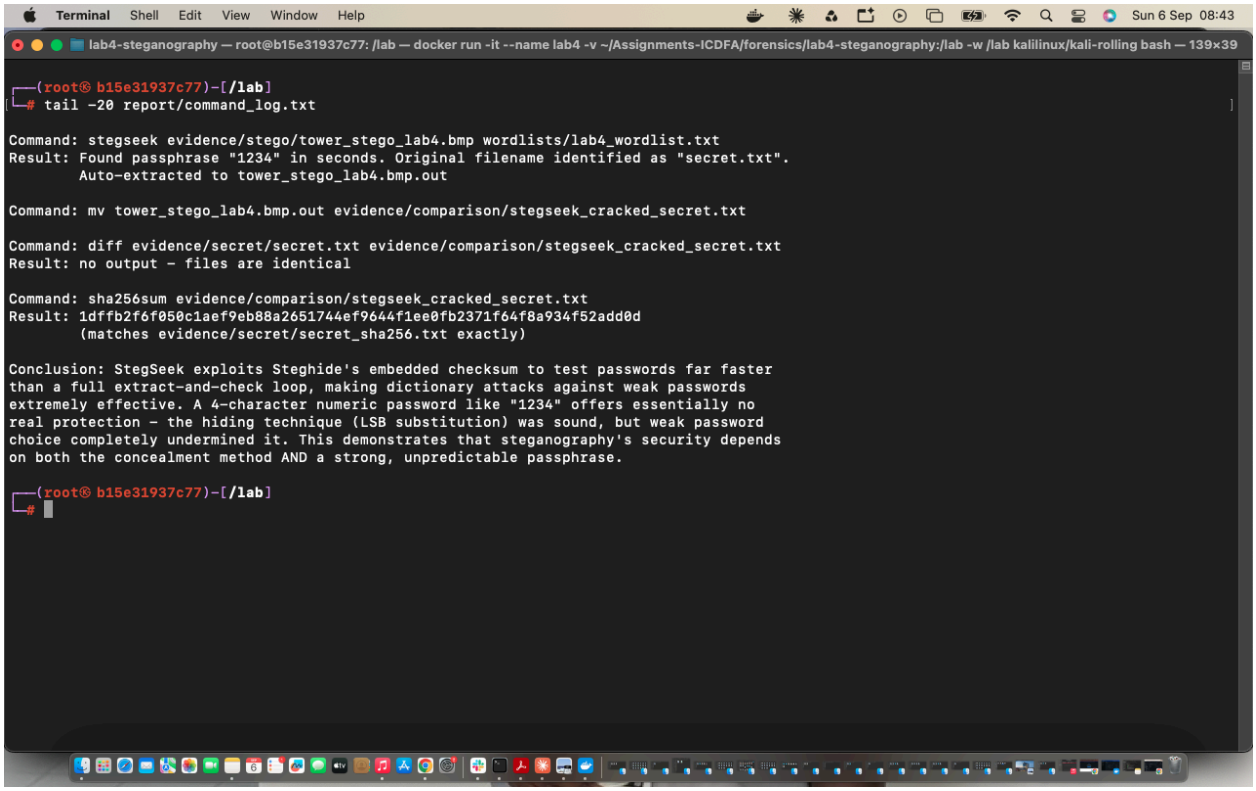
```
$ diff secret.txt stegseek_cracked_secret.txt
(no output)

$ sha256sum stegseek_cracked_secret.txt
1dffb2f6f050c1aef9eb88a2651744ef9644f1ee0fb2371f64f8a934f52add0d
```

Identical match. So the hiding technique itself is solid, but a short, guessable password like “1234” gave essentially zero real protection. This was probably the biggest single takeaway from the whole lab: how you hide something and how you protect it with a password are two completely separate problems, and both have to be strong for the data to actually be safe.


Terminal Shell Edit View Window Help
Sun 6 Sep 08:17
lab4-steganography — root@b15e31937c77: /lab — docker run -it --name lab4 -v ~/Assignments-ICDFA/forensics/lab4-steganography:/lab -w /lab kalilinux/kali-rolling bash — 139×39
bash: man: command not found
┌──(root㉿b15e31937c77)-[/lab]
└─# cat >> report/command_log.txt << 'EOF'
> === Part E: Comparison (Original vs Stego) ===
Command: ls -l evidence/original/tower_original_image_for_lab.bmp evidence/stego/tower_stego_lab4.bmp
Result: identical size (9,277,494 bytes) - LSB substitution does not change file size
Command: sha256sum evidence/original/tower_original_image_for_lab.bmp evidence/stego/tower_stego_lab4.bmp
Result: hashes differ - confirms pixel data was modified
Command: xxd -l 64 evidence/original/tower_original_image_for_lab.bmp
xxd -l 64 evidence/stego/tower_stego_lab4.bmp
Result: identical header bytes - BMP header/metadata untouched, only pixel data changed
Command: diff <(strings evidence/original/tower_original_image_for_lab.bmp) <(strings evidence/stego/tower_stego_lab4.bmp) | wc -l
Result: 2428 differing lines - confirms widespread byte-level changes
Command: strings evidence/stego/tower_stego_lab4.bmp | grep -i "Abubakari"
Result: empty - hidden message not visible as plaintext (Steghide encrypts payload before embedding)
Conclusion: Steganography via LSB substitution preserves file size and header metadata,
alters pixel-level data throughout the file, and hides an encrypted payload that is not
detectable via plain string/text inspection.
EOF
┌──(root㉿b15e31937c77)-[/lab]
└─# tail -20 report/command_log.txt
=== Part E: Comparison (Original vs Stego) ===
Command: ls -l evidence/original/tower_original_image_for_lab.bmp evidence/stego/tower_stego_lab4.bmp
Result: identical size (9,277,494 bytes) - LSB substitution does not change file size
Command: sha256sum evidence/original/tower_original_image_for_lab.bmp evidence/stego/tower_stego_lab4.bmp
Result: hashes differ - confirms pixel data was modified
Command: xxd -l 64 evidence/original/tower_original_image_for_lab.bmp
xxd -l 64 evidence/stego/tower_stego_lab4.bmp
Result: identical header bytes - BMP header/metadata untouched, only pixel data changed


Terminal Shell Edit View Window Help
Sun 6 Sep 08:38
lab4-steganography — root@b15e31937c77: /lab — docker run -it --name lab4 -v ~/Assignments-ICDFA/forensics/lab4-steganography:/lab -w /lab kalilinux/kali-rolling bash — 139×39
┌──(root㉿b15e31937c77)-[/lab]
└─# cat > wordlists/lab4_wordlist.txt << 'EOF'
password
letmein
qwerty
1234
admin123
forensics
steganography
2026lab
EOF
┌──(root㉿b15e31937c77)-[/lab]
└─# cat wordlists/lab4_wordlist.txt
password
letmein
qwerty
1234
admin123
forensics
steganography
2026lab
┌──(root㉿b15e31937c77)-[/lab]
└─# stegseek evidence/stego/tower_stego_lab4.bmp wordlists/lab4_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek
[i] Found passphrase: "1234"
[i] Original filename: "secret.txt".
[i] Extracting to "tower_stego_lab4.bmp.out".
┌──(root㉿b15e31937c77)-[/lab]
└─#


Terminal Shell Edit View Window Help
Sun 6 Sep 08:41
lab4-steganography — root@b15e31937c77: /lab — docker run -it --name lab4 -v ~/Assignments-ICDFA/forensics/lab4-steganography:/lab -w /lab kalilinux/kali-rolling bash — 139×39
┌──(root㉿ b15e31937c77)-[/lab]
└─# mv tower_stego_lab4.bmp.out evidence/comparison/stegseek_cracked_secret.txt
┌──(root㉿ b15e31937c77)-[/lab]
└─# diff evidence/secret/secret.txt evidence/comparison/stegseek_cracked_secret.txt
sha256sum evidence/comparison/stegseek_cracked_secret.txt
1dffb2f6f050c1aef9eb88a2651744ef9644f1ee0fb2371f64f8a934f52add0d evidence/comparison/stegseek_cracked_secret.txt
┌──(root㉿ b15e31937c77)-[/lab]
└─# diff evidence/secret/secret.txt evidence/comparison/stegseek_cracked_secret.txt
┌──(root㉿ b15e31937c77)-[/lab]
└─# sha256sum evidence/comparison/stegseek_cracked_secret.txt
1dffb2f6f050c1aef9eb88a2651744ef9644f1ee0fb2371f64f8a934f52add0d evidence/comparison/stegseek_cracked_secret.txt
┌──(root㉿ b15e31937c77)-[/lab]
└─#


Terminal Shell Edit View Window Help
Sun 6 Sep 08:43
lab4-steganography — root@b15e31937c77: /lab — docker run -it --name lab4 -v ~/Assignments-ICDFA/forensics/lab4-steganography:/lab -w /lab kalilinux/kali-rolling bash — 139×39
┌──(root㉿ b15e31937c77)-[/lab]
└─# tail -20 report/command_log.txt
Command: stegseek evidence/stego/tower_stego_lab4.bmp wordlists/lab4_wordlist.txt
Result: Found passphrase "1234" in seconds. Original filename identified as "secret.txt".
Auto-extracted to tower_stego_lab4.bmp.out
Command: mv tower_stego_lab4.bmp.out evidence/comparison/stegseek_cracked_secret.txt
Command: diff evidence/secret/secret.txt evidence/comparison/stegseek_cracked_secret.txt
Result: no output - files are identical
Command: sha256sum evidence/comparison/stegseek_cracked_secret.txt
Result: 1dffb2f6f050c1aef9eb88a2651744ef9644f1ee0fb2371f64f8a934f52add0d
(matches evidence/secret/secret_sha256.txt exactly)
Conclusion: StegSeek exploits Steghide's embedded checksum to test passwords far faster
than a full extract-and-check loop, making dictionary attacks against weak passwords
extremely effective. A 4-character numeric password like "1234" offers essentially no
real protection - the hiding technique (LSB substitution) was sound, but weak password
choice completely undermined it. This demonstrates that steganography's security depends
on both the concealment method AND a strong, unpredictable passphrase.
┌──(root㉿ b15e31937c77)-[/lab]
└─#

Part G – Doing the Whole Thing Again On My Own

For the last part, I repeated the entire process from scratch using my own message and my own choices to prove to myself that I actually understood the steps rather than just following a script.

My own note (my_hidden_note.txt)

> *Full Name: Abubakari-Sadique Hamidu*
> *Part G Independent Note: This file was created independently for ICDFA Lab 4, Part G, to demonstrate the steganography and password-cracking workflow without following a provided script. Steganography hides the existence of a message inside another file, unlike encryption, which hides only its content. In digital forensics, investigators must know how to detect steganography because it can be used to smuggle stolen data or malicious payloads inside innocent-looking images, and how to test password strength, because a weak passphrase can be cracked in seconds with tools like StegSeek.*

Embedding it

```
$ sha256sum my_hidden_note.txt
8d5a5066246946a94eec3adb221f6269d07606686fc7ea2a9a1f980c9c56f1b3

$ steghide embed -ef my_hidden_note.txt -cf tower_original_image_for_lab.bmp -sf
tower_stego_partG.bmp -p 1234

$ sha256sum tower_stego_partG.bmp
934c92b9a2a338eb3ef7630cab63f351c839c16f8b9cf2c2d1511c6124c8f8b5
```

Extracting and checking it

```
$ steghide extract -sf tower_stego_partG.bmp -xf extracted_partG.txt -p 1234

$ diff my_hidden_note.txt extracted_partG.txt
(no output)

$ sha256sum extracted_partG.txt
8d5a5066246946a94eec3adb221f6269d07606686fc7ea2a9a1f980c9c56f1b3
```

Same outside comparison as Part E

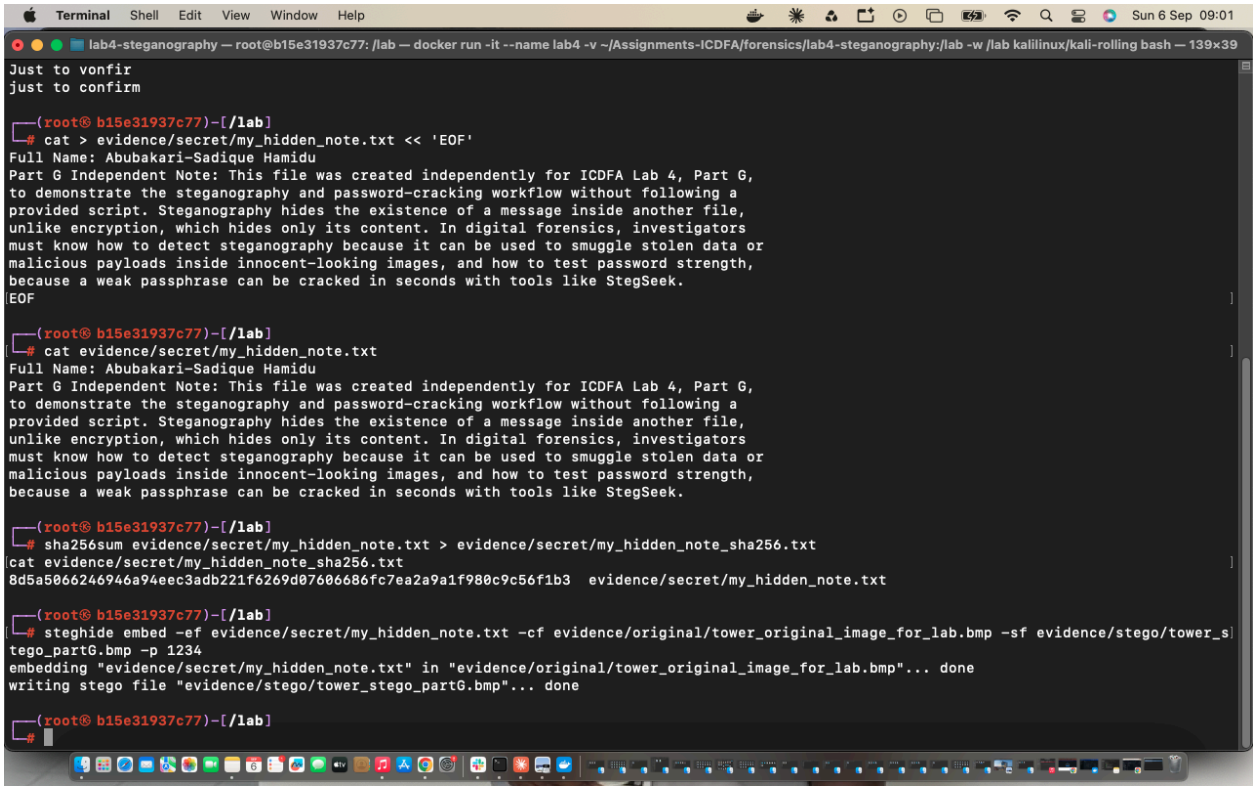
```
$ ls -l tower_original_image_for_lab.bmp tower_stego_partG.bmp
Both files: 9,277,494 bytes — same size

$ sha256sum tower_original_image_for_lab.bmp tower_stego_partG.bmp
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db
tower_original_image_for_lab.bmp
934c92b9a2a338eb3ef7630cab63f351c839c16f8b9cf2c2d1511c6124c8f8b5  tower_stego_partG.bmp
```

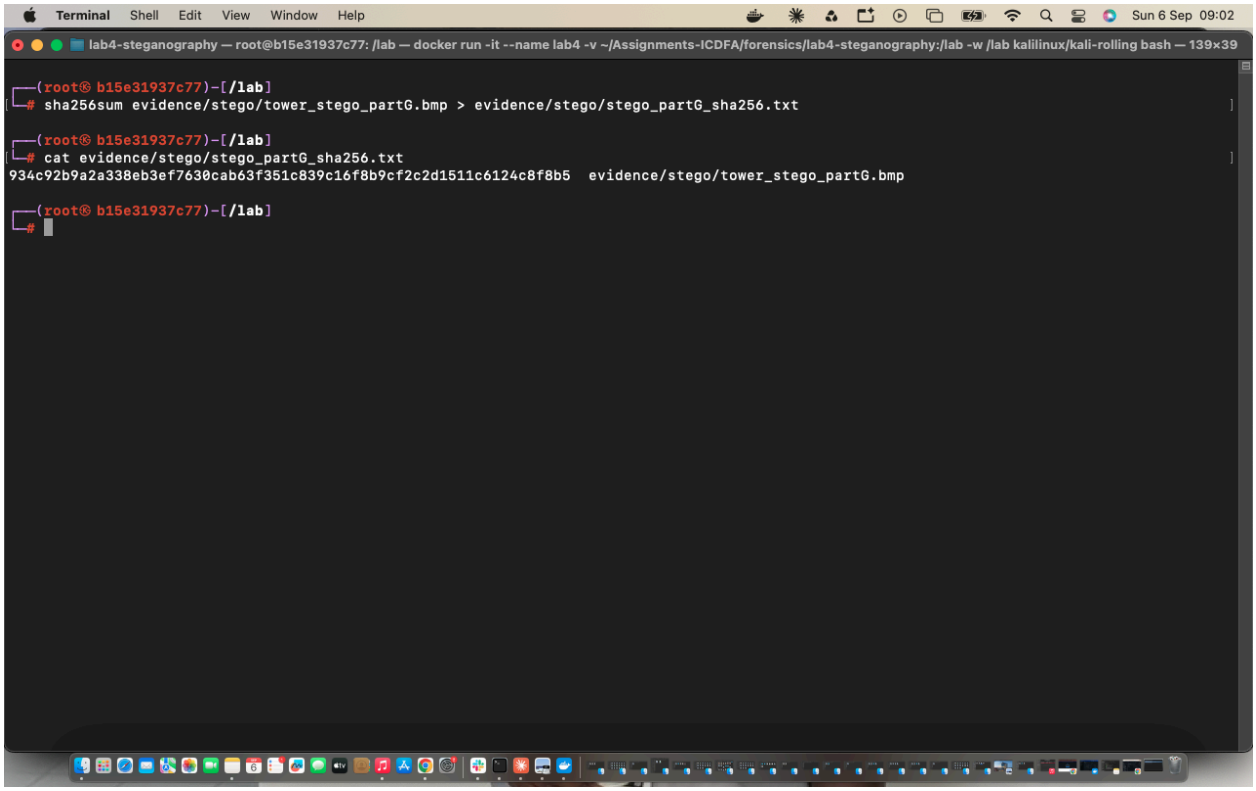
Exact same pattern as before — same size, different hash.

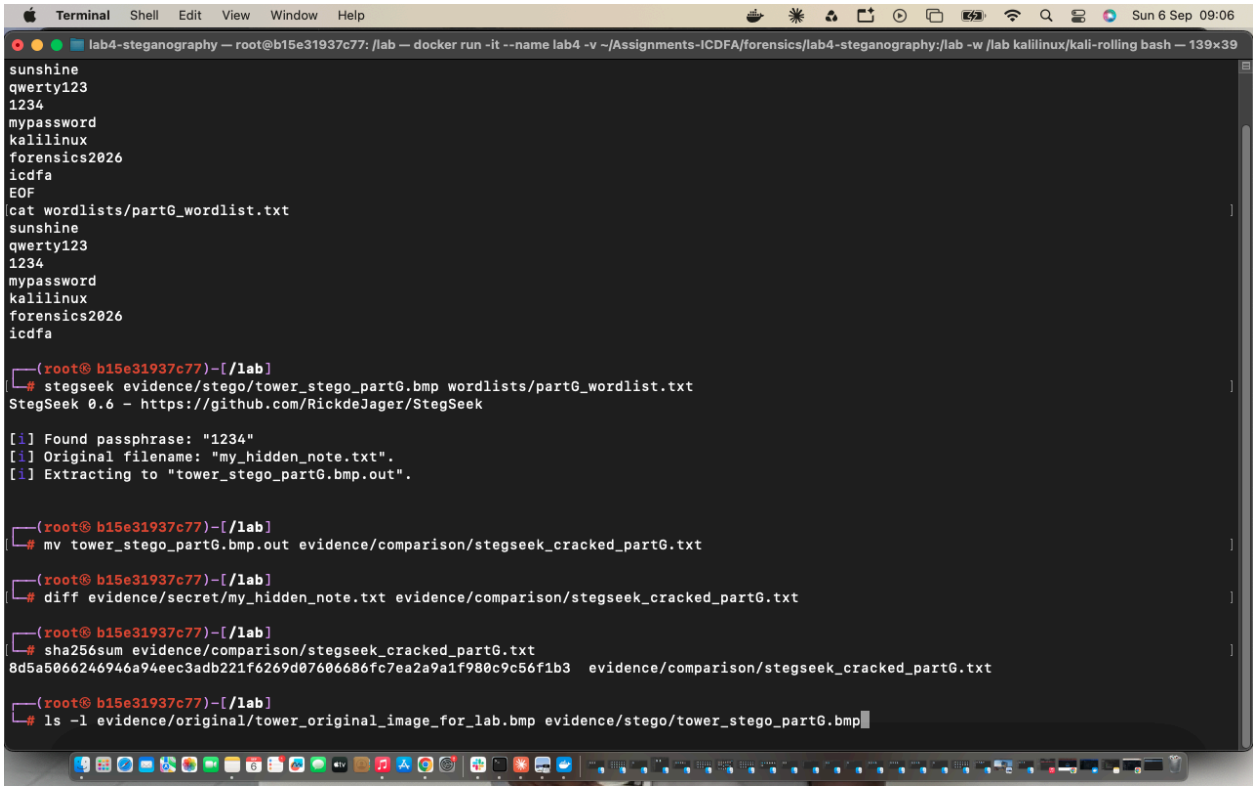
My own wordlist and StegSeek run

```
sunshine
qwerty123
1234
mypassword
kalilinux
forensics2026
icdfa
$ stegseek tower_stego_partG.bmp partG_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "1234"
[i] Original filename: "my_hidden_note.txt".
[i] Extracting to "tower_stego_partG.bmp.out".

$ diff my_hidden_note.txt stegseek_cracked_partG.txt
(no output)

$ sha256sum stegseek_cracked_partG.txt
8d5a5066246946a94eec3adb221f6269d07606686fc7ea2a9a1f980c9c56f1b3
```

Cracked instantly again, and the recovered file matched my original note exactly. Going through the whole thing a second time on my own, and getting the same kind of result, is what actually convinced me I understood it. The first time through I was mostly following instructions step by step; this time I knew what each command was supposed to do and why, before I even ran it.


Terminal Shell Edit View Window Help
Sun 6 Sep 09:01
lab4-steganography — root@b15e31937c77: /lab — docker run -it --name lab4 -v ~/Assignments-ICDFA/forensics/lab4-steganography:/lab -w /lab kalilinux/kali-rolling bash — 139×39
Just to vonfir
just to confirm
┌──(root㉿ b15e31937c77)-[/lab]
└─# cat > evidence/secret/my_hidden_note.txt << 'EOF'
Full Name: Abubakari-Sadique Hamidu
Part G Independent Note: This file was created independently for ICDFA Lab 4, Part G,
to demonstrate the steganography and password-cracking workflow without following a
provided script. Steganography hides the existence of a message inside another file,
unlike encryption, which hides only its content. In digital forensics, investigators
must know how to detect steganography because it can be used to smuggle stolen data or
malicious payloads inside innocent-looking images, and how to test password strength,
because a weak passphrase can be cracked in seconds with tools like StegSeek.
EOF
┌──(root㉿ b15e31937c77)-[/lab]
└─# cat evidence/secret/my_hidden_note.txt
Full Name: Abubakari-Sadique Hamidu
Part G Independent Note: This file was created independently for ICDFA Lab 4, Part G,
to demonstrate the steganography and password-cracking workflow without following a
provided script. Steganography hides the existence of a message inside another file,
unlike encryption, which hides only its content. In digital forensics, investigators
must know how to detect steganography because it can be used to smuggle stolen data or
malicious payloads inside innocent-looking images, and how to test password strength,
because a weak passphrase can be cracked in seconds with tools like StegSeek.
┌──(root㉿ b15e31937c77)-[/lab]
└─# sha256sum evidence/secret/my_hidden_note.txt > evidence/secret/my_hidden_note_sha256.txt
cat evidence/secret/my_hidden_note_sha256.txt
8d5a5066246946a94eec3adb221f6269d07606686fc7ea2a9a1f980c9c56f1b3 evidence/secret/my_hidden_note.txt
┌──(root㉿ b15e31937c77)-[/lab]
└─# steghide embed -ef evidence/secret/my_hidden_note.txt -cf evidence/original/tower_original_image_for_lab.bmp -sf evidence/stego/tower_s
tego_partG.bmp -p 1234
embedding "evidence/secret/my_hidden_note.txt" in "evidence/original/tower_original_image_for_lab.bmp"... done
writing stego file "evidence/stego/tower_stego_partG.bmp"... done
┌──(root㉿ b15e31937c77)-[/lab]
└─#


Terminal Shell Edit View Window Help
Sun 6 Sep 09:02
lab4-steganography — root@b15e31937c77: /lab — docker run -it --name lab4 -v ~/Assignments-ICDFA/forensics/lab4-steganography:/lab -w /lab kalilinux/kali-rolling bash — 139×39
┌──(root㉿ b15e31937c77)-[/lab]
└─# sha256sum evidence/stego/tower_stego_partG.bmp > evidence/stego/stego_partG_sha256.txt
┌──(root㉿ b15e31937c77)-[/lab]
└─# cat evidence/stego/stego_partG_sha256.txt
934c92b9a2a338eb3ef7630cab63f351c839c16f8b9cf2c2d1511c6124c8f8b5 evidence/stego/tower_stego_partG.bmp
┌──(root㉿ b15e31937c77)-[/lab]
└─#



Conclusion

This lab showed me that steganography is a genuinely different kind of security from encryption. Encryption scrambles a message so it can't be read; steganography hides the fact that there's a message there at all. Used together, the way Steghide does it, you get both, but only if the password holding it all together is actually strong. A weak password like 1234 made StegSeek's job almost instant, while the hiding method itself (changing pixel bits one at a time) held up completely fine against size checks, header checks, and text scanning.

If I were doing this for real casework, the big lesson I'm taking away is: don't judge a file by its size or its header, don't assume something is safe just because a quick scan doesn't turn anything up, and never trust a short, common password to protect anything that actually matters.